

# Capacitação Back-end — apostila completa

AUTOMIC JR. · QUATRO ENCONTROS

# Sumário

---

## Aula 0 — Preparação

1. Se você usa Windows: instale o WSL2 primeiro
2. Git
3. Ruby 3.4.9 via mise
  - Instalar o mise
  - Instalar as dependências de compilação do Ruby
  - Instalar o Ruby
  - Instalar o Rails
4. Docker
5. VS Code e extensões
6. Insomnia (ou Postman)
7. Contas
  - GitHub
  - Azure for Students — faça isso com antecedência

Checagem final

Como a capacitação funciona

Se algo deu errado

## Aula 1 — O que é back-end, Ruby e o primeiro Rails

1. Os dois lados
2. A rede, o mínimo necessário
  - O que é um servidor, afinal
  - IP e porta
  - DNS
  - Anatomia de uma URL
3. HTTP
  - Como ela é, por dentro
  - Cabeçalhos
  - Content-Type importa
  - Os métodos
  - Os status
  - HTTP não tem memória
  - Veja funcionando

#### 4. API REST

#### 5. Ruby, partindo do Python

Tudo é objeto

Símbolos

Blocos

? e !

Argumentos nomeados, e o atalho do Ruby 3.1

Struct

Exceções

Módulos e mixins

Dependências

Prática rápida

#### 6. Rails

As duas leis

Rails × Django

MVC

Zeitwerk: o nome diz o caminho

Os três ambientes

#### 7. Mão na massa

Criar o projeto

Subir o banco

Preparar e subir a aplicação

O tour das pastas

Os comandos do dia a dia

#### 8. A primeira rota

Teste

#### Exercício da aula

1. Crie o seu projeto

2. Publique no seu GitHub

3. Suba o banco

4. Suba a aplicação

5. Escreva a rota

6. Confira

7. Escreva o teste

8. Commit

#### Recapitulando

## Aula 2 — Banco de dados, ActiveRecord e o model User

1. Por que Postgres, e não SQLite  
Transação
2. ActiveRecord  
ActiveSupport: os métodos que o Rails inventou
3. Migrations  
schema.rb  
Migrations são incrementais
4. Senha: o que nunca fazer  
Hash não é criptografia  
Por que bcrypt e não SHA-256  
has\_secure\_password
5. Validações  
Regra de ouro: normalize antes de validar
6. Associações  
Quem guarda a chave  
A migration  
Os dois lados  
Usando  
A armadilha N+1
7. Concerns — o mixin da Aula 1, na prática  
Três decisões dentro do concern, e o porquê de cada uma
8. O console
9. Seeds e fixtures
10. Testando o model
11. Uma sutileza de segurança: authenticate\_by\_email  
Exercício da aula  
Recapitulando

## Aula 3 — As rotas de autenticação

1. O caminho de uma requisição
2. A arquitetura de pastas  
Por que service object  
Por que serializer  
Um formato de erro só  
before\_action: o filtro

A pilha de middleware

Strong parameters

### 3. Rota 1 — Cadastro

A decisão que não é óbvia

### 4. E-mail: o mailer

deliver\_now e não deliver\_later

### 5. Ativação da conta

### 6. O problema: HTTP não tem memória

JWT

O detalhe que quebra tudo

### 7. Rota 2 — Login

O que é um cookie

XSS

CSRF

401 × 403

A mesma mensagem para os dois erros

### 8. Rota 3 — Logout, o problema difícil

a) Apagar no cliente

b) Denylist por jti — revoga aquele token

c) token\_version — derruba tudo de uma vez

A verificação completa

### 9. Rota 4 — Recuperação de senha

request sempre responde 200

confirm e a transação

### 10. Testando

As ferramentas de qualidade

### 11. CORS

### 12. O fluxo inteiro, no terminal

Exercício da aula

Recapitulando

## Aula 4 — VPS, Docker, Kamal e deploy

### 1. O que é uma VPS

Por que VM com containers, e não Kubernetes

## 2. Criar a VM na Azure

Cotas antes de tudo

O assistente

Antes: duas chaves, um par

A chave privada

Abrir o SSH só para você

Primeiro acesso

## 3. Preparar o Ubuntu

Atualizar

Docker, do repositório oficial

Root e permissões

Swap

Endurecer o SSH

## 4. Docker

Imagem × container

O Dockerfile do Rails

.dockerignore

Registry

## 5. Kamal

config/deploy.yml, por partes

As três camadas de segredo

## 6. Segredos do Rails

credentials e master.key

Antes: o que é uma variável de ambiente

Credentials × ENV

## 7. GitHub Actions

CI: o portão

Deploy: o interessante

## 8. OIDC — por que não usar senha

Criando

Cadastrando no GitHub

## 9. Cloudflare e TLS

HTTPS é HTTP dentro de um túnel

O handshake, em três passos

Certificado e autoridade

Proxy reverso

DNS

Certificado Origin CA

Full (strict)

Por que não Let's Encrypt

10. O primeiro deploy

11. Operar

Backup

Exercício da aula

Recapitulando

O que o seem-backend tem a mais

## **Ruby para quem sabe Python**

Sintaxe

Só em Ruby

Aspas: a diferença que Python não tem

Números: a pegadinha da divisão

Condicionais e case

Ternário e loops

puts, print e p

require × import

Splat

Estruturas de dados

Símbolos

Blocos, o coração do Ruby

Bloco, proc e lambda

Equivalências

Argumentos nomeados

O atalho do Ruby 3.1

Struct

Exceções

Atalhos de literal

Classes

? e !

Módulos e mixins

Dependências e ferramentas

Rails × Django

Consultas lado a lado

Pegadinhas de quem vem do Python

## Glossário

### Troubleshooting

#### Ambiente

mise: command not found

A instalação do Ruby falha compilando

gem install rails reclama de permissão

permission denied ao rodar docker

No Windows, docker não aparece dentro do Ubuntu

#### Banco

address already in use ao subir o compose

PG::ConnectionBad: could not connect to server

PG::UndefinedTable ou PendingMigrationError

Quero começar do zero

#### Testes

Um teste passa sozinho e falha junto com os outros

O teste de logout passa mas a revogação não funciona

Bullet::Notification::UnoptimizedQueryError

#### Autenticação

Sempre 401, mesmo com o token certo

O e-mail não chega em desenvolvimento

ActionController::ParameterMissing (400)

422 no cadastro sem dizer o motivo direito

#### Azure e VM

NotAvailableForSubscription ao criar a VM

SSH dá timeout

Permission denied (publickey)

Perdi o acesso depois de mexer no sshd

#### Deploy

OIDC: AADSTS700213

az: command not found ou falha transitória do Azure CLI

A regra temporária ficou no NSG

Kamal: Docker is not installed

A aplicação sobe mas não acha o banco

Cloudflare 521 / 522

Cloudflare 525 / 526

Let's Encrypt falha atrás do Cloudflare

SMTP dá timeout na VM

O deploy passa mas o site responde 500

Preciso voltar atrás agora

# Aula 0 — Preparação

---

**Faça isso antes do primeiro encontro.** Instalar Ruby, Docker e criar contas leva tempo, e é o que mais atrasa capacitação. Se travar em algum passo, manda mensagem no grupo — não espere o dia.

Ao final você deve conseguir rodar os quatro comandos da seção [Checagem final](#).

---

## 1. Se você usa Windows: instale o WSL2 primeiro

---

Ruby e Rails rodam mal no Windows nativo. A forma suportada é o **WSL2** (um Linux de verdade dentro do Windows). Todo o resto deste guia você fará **dentro do WSL2**, não no PowerShell.

No PowerShell **como administrador**:

```
wsl --install -d Ubuntu-24.04
```

Reinicie o computador. Ao abrir o Ubuntu pela primeira vez, ele pede um usuário e uma senha — anote a senha, ela é o seu `sudo`.

A partir daqui, "terminal" significa **o terminal do Ubuntu (WSL2)**.

Se você usa Linux ou macOS, pule esta seção.

---

## 2. Git

---

```
# Ubuntu / WSL2
sudo apt update && sudo apt install -y git curl build-essential

# macOS (instala as ferramentas de linha de comando da Apple, que incluem o git)
xcode-select --install
```

Configure seu nome e e-mail — eles vão em todo commit que você fizer:

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
```

---

### 3. Ruby 3.4.9 via mise

O `mise` é um gerenciador de versões: ele instala a versão exata de Ruby que o projeto pede, sem mexer no Ruby do sistema. É o equivalente do `pyenv` do mundo Python.

**Por que não `apt install ruby`?** Porque a versão do sistema é antiga e compartilhada. Se dois projetos pedirem versões diferentes, você fica sem saída. `mise` resolve isso por projeto.

#### Instalar o mise

```
curl https://mise.run | sh
echo 'eval "$( ~/.local/bin/mise activate bash )"' >> ~/.bashrc
exec bash
```

Usa `zsh` (padrão do macOS)? Troque as duas ocorrências de `bash` por `zsh` e o `~/.bashrc` por `~/.zshrc`.

#### Instalar as dependências de compilação do Ruby

```
# Ubuntu / WSL2
sudo apt install -y autoconf patch rustc libssl-dev libyaml-dev libreadline-dev \
  zlib1g-dev libgmp-dev libncurses-dev libffi-dev libgdbm-dev libdb-dev uuid-dev \
  libpq-dev libvips

# macOS (precisa do Homebrew: https://brew.sh)
brew install openssl@3 readline libyaml gmp libpq vips
```

#### Instalar o Ruby

```
mise use --global ruby@3.4.9
```

Isso compila o Ruby do zero — leva de **5 a 15 minutos**. É normal.

```
ruby -v      # ruby 3.4.9 ...
gem -v
```

#### Instalar o Rails

```
gem install rails -v 8.1.3
rails -v      # Rails 8.1.3
```

## 4. Docker

O banco de dados (PostgreSQL) vai rodar dentro de um container Docker, em vez de instalado na sua máquina. Assim todo mundo tem exatamente a mesma versão, e desinstalar é apagar um container.

- **Windows/WSL2 e macOS:** instale o [Docker Desktop](#). No Windows, nas configurações, habilite a integração com a distro `Ubuntu-24.04`.
- **Linux:** siga o [guia oficial](#) e depois rode `sudo usermod -aG docker $USER` e **saia e entre de novo na sessão**.

Testando:

```
docker run --rm hello-world
```

Se aparecer "Hello from Docker!", está pronto.

## 5. VS Code e extensões

Baixe em [code.visualstudio.com](https://code.visualstudio.com). No Windows, instale no **Windows** (não dentro do WSL) — ele se conecta ao WSL sozinho.

Extensões (Ctrl+Shift+X e busque pelo nome):

| Extensão                  | Para quê                                                     |
|---------------------------|--------------------------------------------------------------|
| <b>Ruby LSP</b> (Shopify) | Autocomplete, ir para definição, erros em tempo real         |
| <b>Ruby Rails</b> (Aki)   | Navegação entre model/controller/view                        |
| <b>Docker</b> (Microsoft) | Ver e gerenciar containers pela barra lateral                |
| <b>WSL</b> (Microsoft)    | <i>Só Windows</i> — abrir projetos do Ubuntu no VS Code      |
| <b>GitLens</b>            | Ver quem escreveu cada linha e quando                        |
| <b>vscode-icons</b>       | Ícones por tipo de arquivo (a mesma da capacitação de front) |

## 6. Insomnia (ou Postman)

Nossa API não tem tela — ela responde JSON. Para chamar as rotas, usaremos um cliente HTTP. Baixe o [Insomnia](#) (mais simples) ou o [Postman](#), o que preferir.

## 7. Contas

### GitHub

Crie em [github.com](https://github.com) se ainda não tem. Depois **ative o GitHub Student Pack** em [education.github.com](https://education.github.com) com seu e-mail `@aluno.ufop.edu.br` — a aprovação pode levar alguns dias, e você ganha créditos e ferramentas de graça.

Instale o **GitHub CLI**, que na Aula 1 publica o seu projeto num comando e na Aula 4 cadastra os segredos do deploy:

```
# Ubuntu / WSL2
sudo apt install -y gh
# macOS
brew install gh

gh auth login          # escolha GitHub.com → HTTPS → login pelo navegador
gh auth status         # tem que dizer "Logged in to github.com"
```

Configure também o acesso por SSH ao GitHub, se ainda não tem:

```
ssh-keygen -t ed25519 -C "seu@email.com"      # Enter em tudo
gh ssh-key add ~/.ssh/id_ed25519.pub --title "meu-notebook"
ssh -T git@github.com                          # tem que cumprimentar você pelo nome
```

### Azure for Students — faça isso com antecedência

Na Aula 4 cada um vai subir a própria máquina virtual na nuvem. Usaremos o [Azure for Students](https://azure.microsoft.com/en-us/free/students/): **US\$ 100 de crédito, sem cartão de crédito**, mediante e-mail institucional.

1. Acesse o link e clique em **Comece gratuitamente**.
2. Entre com seu e-mail institucional (`@aluno.ufop.edu.br`).
3. Confirme a verificação acadêmica.
4. Entre no [portal.azure.com](https://portal.azure.com) e confirme que o crédito aparece.

A verificação **pode demorar dias**. Faça agora, não na véspera da Aula 4.

### Checagem final

Cole os quatro comandos no terminal. Se todos responderem, você está pronto:

```
ruby -v          # ruby 3.4.9
rails -v         # Rails 8.1.3
docker -v        # Docker version 2x.x.x
git --version
gh auth status   # Logged in to github.com
```

## Como a capacitação funciona

Você vai trabalhar em **dois diretórios**:

```
~/capacitacao-gabarito/  ← o repositório da capacitação. Só leitura.
~/automic_auth_api/      ← o SEU projeto. Criado na Aula 1, é onde você escreve.
```

Clone o gabarito já:

```
git clone <url-do-repositório> ~/capacitacao-gabarito
```

O seu projeto você cria no primeiro encontro, com o passo a passo de [01-fundamentos.md](#). Ele precisa ficar no **seu** GitHub: na Aula 4, a credencial que autoriza o deploy é emitida para `repo:SEU-USUARIO/SEU-REPO`.

## Se algo deu errado

| Sintoma                                                        | Provável causa e saída                                                                                                                                                |
|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>mise: command not found</code>                           | A linha do <code>activate</code> não foi para o arquivo certo. Confira <code>~/.bashrc</code> (ou <code>~/.zshrc</code> ) e rode <code>exec bash</code> .             |
| A instalação do Ruby falha com erro de compilação              | Faltou alguma dependência do passo 3. Rode o <code>apt install / brew install</code> de novo e tente <code>mise install ruby@3.4.9</code> outra vez.                  |
| <code>permission denied</code> no <code>docker</code>          | Você não está no grupo <code>docker</code> . Rode <code>sudo usermod -aG docker \$USER</code> e <b>saia e entre de novo</b> (só reabrir o terminal não basta).        |
| No Windows, o <code>docker</code> não aparece dentro do Ubuntu | Docker Desktop → Settings → Resources → WSL Integration → habilite a distro <code>Ubuntu-24.04</code> .                                                               |
| <code>gem install rails</code> reclama de permissão            | Você está usando o Ruby do sistema, não o do <code>mise</code> . Confira com <code>which ruby</code> — deve apontar para dentro de <code>~/.local/share/mise</code> . |

Mais casos em [troubleshooting.md](#).

# Aula 1 — O que é back-end, Ruby e o primeiro Rails

**Duração:** ~3h · **Você sai daqui com:** uma API respondendo `GET /api/v1/status`. **Gabarito:** `cd ~/capitacao-gabarito && git checkout aula-01`

Pré-requisitos instalados em `00-preparacao.md`.

## 1. Os dois lados

Na capacitação de front-end você aprendeu a fazer o que o usuário vê. Back-end é o outro lado.

| Front-end                           | Back-end                            |
|-------------------------------------|-------------------------------------|
| A parte visível, que o usuário toca | A parte lógica, nos bastidores      |
| HTML, CSS, JavaScript               | Ruby, Python, Java, Go, Node        |
| Roda no navegador ou no celular     | Roda num servidor, longe do usuário |
| Se cair, a tela quebra              | Se cair, nada funciona              |

O back-end guarda os dados, decide quem pode ver o quê, aplica as regras de negócio, autentica usuários e conversa com outros sistemas (e-mail, pagamento, mapas).

**Por que isso não pode viver no front?** Porque tudo que roda no navegador o usuário consegue ler e alterar. A validação de senha no front é conveniência; a que vale é a do back.

**A analogia:** o salão do restaurante é o front-end — mesa, cardápio, garçom. A cozinha é o back-end: ninguém entra, mas é lá que a comida sai. O pedido é a requisição, o prato é a resposta. Você não pede pra ver a receita; você pede o prato.

## 2. A rede, o mínimo necessário

### O que é um servidor, afinal

Não é uma máquina especial. É um **programa esperando**: ele fica ligado, escutando numa porta, e responde ao que chegar ali. `bin/rails server` sobe esse programa na porta 3000 — na sua máquina ou numa VM na nuvem, é o mesmo programa.

## IP e porta

**IP** é o endereço da máquina ( `57.156.65.151` ). **Porta** é a sala dentro dela: um número de 1 a 65535. Uma máquina tem milhares de portas, e um programa diferente pode escutar em cada uma.

| Porta | Quem mora ali            |
|-------|--------------------------|
| 22    | SSH                      |
| 80    | HTTP                     |
| 443   | HTTPS                    |
| 5432  | Postgres                 |
| 3000  | Rails em desenvolvimento |

`127.0.0.1` — o `localhost` — significa sempre "esta máquina aqui". Na Aula 4 vamos abrir e fechar portas na mão, no firewall da Azure.

## DNS

Ninguém decora `57.156.65.151`. O **DNS** é a agenda telefônica da internet: você digita `api.seemxxiii.tech` e alguém pergunta ao DNS qual é o IP.

A resposta fica em cache por um tempo — o **TTL**. É por isso que apontar um domínio para outro servidor não vale na hora. Na Aula 4 você vai criar um desses registros.

## Anatomia de uma URL

```
https :// api.seemxxiii.tech : 443 /api/v1/lectures ?page=2 #topo
|         |           |         |         |         |
esquema  host        porta   caminho  query   fragmento
```

- **Esquema** — o protocolo: `http`, `https`, `ssh`, `postgres`.
- **Porta** — opcional. `http` assume 80, `https` assume 443.
- **Fragmento** — nunca vai para o servidor; é só para o navegador.

É por isso que `localhost:3000` tem os dois pontos: a porta não é a padrão.

## 3. HTTP

Cliente é quem pede (navegador, app, outro servidor). Servidor é quem responde. **O cliente sempre começa a conversa** — o servidor nunca liga primeiro.

Uma **requisição** tem método, caminho, cabeçalhos e corpo. Uma **resposta** tem status, cabeçalhos e corpo.

## Como ela é, por dentro

```
POST /api/v1/sessions HTTP/1.1      ← método, caminho, versão
Host: api.seemxxiii.tech             ]
Content-Type: application/json        | cabeçalhos
Accept: application/json              ]
                                     ← linha em branco separa
{"email": "ana@ufop.br", "password": "..."} ← corpo
```

É **texto puro**. HTTP é literalmente isso trafegando num cano.

## Cabeçalhos

São **metadados**: informação *sobre* a mensagem, não a mensagem. Um par `chave: valor` por linha.

| Cabeçalho                                     | Para quê                                       |
|-----------------------------------------------|------------------------------------------------|
| <code>Content-Type</code>                     | Em que formato vai o corpo                     |
| <code>Accept</code>                           | Em que formato eu quero a resposta             |
| <code>Authorization</code>                    | Quem sou eu — vai ser o nosso token, na Aula 3 |
| <code>Set-Cookie</code> / <code>Cookie</code> | Como o navegador guarda estado                 |
| <code>User-Agent</code>                       | Que programa está chamando                     |

### `Content-Type` importa

O mesmo dado, dois formatos:

```
application/json      → {"email": "ana@ufop.br"}
application/x-www-form-urlencoded → email=ana%40ufop.br
```

Sem o cabeçalho, o servidor não sabe como ler o corpo. Esquecer isso no `curl` é o erro nº 1 de quem começa: vem `400` e a mensagem não ajuda em nada. Por isso todo `curl` deste material tem `-H 'Content-Type: application/json'`.

## Os métodos

| Método                                | Significa                  |
|---------------------------------------|----------------------------|
| <code>GET</code>                      | Me dá isso. Não muda nada. |
| <code>POST</code>                     | Cria isso.                 |
| <code>PUT</code> / <code>PATCH</code> | Altera isso.               |
| <code>DELETE</code>                   | Apaga isso.                |

O método é uma promessa. Um `GET` que apaga registro é bug, não criatividade.

## Os status

| Faixa | Significa           | Exemplos                                                                                        |
|-------|---------------------|-------------------------------------------------------------------------------------------------|
| 2xx   | Deu certo           | 200 ok, 201 criado                                                                              |
| 3xx   | Foi pra outro lugar | 301, 302                                                                                        |
| 4xx   | <b>Você</b> errou   | 400 pedido mal feito, 401 não autenticado, 403 sem permissão, 404 não existe, 422 dado inválido |
| 5xx   | <b>Nós</b> erramos  | 500 bug nosso                                                                                   |

401 é "quem é você?". 403 é "sei quem você é, e você não pode". Vamos usar os dois na Aula 3.

## HTTP não tem memória

Cada requisição começa do zero. O servidor esquece tudo entre uma e outra.

**Guarde essa frase.** Ela é o problema inteiro da Aula 3: se o servidor esquece tudo, como ele sabe que você já fez login?

## Veja funcionando

```
curl -i https://api.seemxxiii.tech/api/v1/status
```

```
HTTP/2 200
content-type: application/json; charset=utf-8

{"status":"ok","service":"seem-backend"}
```

Isso é uma API real, no ar. É exatamente o que você vai construir.

## 4. API REST

**API** é a porta de entrada do sistema para outros programas. **REST** é um estilo de organizar essa porta: cada coisa do sistema é um **recurso**, com um endereço.

```
/api/v1/lectures    → as palestras
/api/v1/lectures/7  → a palestra 7
```

O que você faz com o recurso é o **verbo**, não o endereço:

|        |                    |            |
|--------|--------------------|------------|
| GET    | /api/v1/lectures   | lista      |
| POST   | /api/v1/lectures   | cria       |
| GET    | /api/v1/lectures/7 | mostra uma |
| DELETE | /api/v1/lectures/7 | apaga      |

`/criarPalestra` · `POST /lectures`

O `v1` no caminho é a versão. Quando a API mudar de forma incompatível, nasce uma `/v2` e a `/v1` continua funcionando — porque o app na loja do celular demora dias para atualizar.

## 5. Ruby, partindo do Python

Ruby foi criado por **Yukihiro Matsumoto (Matz)** em 1995, no Japão, com um objetivo declarado: *fazer o programador feliz*. É interpretada, dinâmica e orientada a objetos — como Python.

A tabela completa está em `ruby-para-pythonistas.md`. O essencial:

```
# Python                                # Ruby
def saudacao(nome, formal=False):      def saudacao(nome, formal: false)
  if formal:                            return "Prezado #{nome}" if formal
    return f"Prezado {nome}"           "Oi, #{nome}"
  return f"Oi, {nome}"                 end
```

Três coisas para reparar:

1. `end` fecha o bloco, em vez da indentação.
2. `if` **no fim da linha** — é o modificador, muito usado em Ruby.
3. **A última linha é o retorno.** Não precisa escrever `return`.

### Tudo é objeto

Em Python você chama `len(x)`. Em Ruby, `x.length`. Não existe função solta: é sempre método de alguém. Até número é objeto — `3.times { puts "oi" }`.

### Símbolos

```
:aluno  :email  :admin
```

Um símbolo é um texto que serve de **etiqueta**, não de conteúdo. O mesmo símbolo é sempre o mesmo objeto na memória. Não existe em Python — lá você usaria uma string.

Regra prática: **conteúdo que o usuário lê** → **string. Nome de coisa no código** → **símbolo**.

## Blocos

Um bloco é um pedaço de código que você entrega para um método executar. É o `lambda` do Python, mas usado o tempo todo.

```
usuarios.each do |u|           # for u in usuarios
  puts u.nome
end

usuarios.map { |u| u.nome }    # [u.nome for u in usuarios]
usuarios.select { |u| u.ativo? } # [u for u in usuarios if u.ativo]
```

Chaves quando cabe numa linha, `do ... end` quando não cabe.

? e !

| Sufixo | Significa                                                             | Exemplo                  |
|--------|-----------------------------------------------------------------------|--------------------------|
| ?      | Devolve verdadeiro ou falso                                           | <code>user.admin?</code> |
| !      | A versão perigosa: estoura erro em vez de devolver <code>false</code> | <code>user.save!</code>  |

É convenção, não regra da linguagem. Mas todo mundo segue.

## Argumentos nomeados, e o atalho do Ruby 3.1

```
def login(email:, password:, client: "mobile")
  end

login(email: "a@b.c", password: "x")
```

Os dois-pontos **depois** do nome tornam obrigatório nomear na chamada. E quando a variável tem o mesmo nome da chave, você omite o valor:

```
Result.new(success?: true, user:)    # user: user
```

Isso aparece em todo service do projeto. **Não é erro de digitação.**

### Struct

Quando você só quer agrupar valores, sem comportamento:

```
Result = Struct.new(:success?, :user, :errors, keyword_init: true)

r = Result.new(success?: true, user: ana)
r.success?    # true
```

É o `namedtuple` / `dataclass` do Python. Todo service deste projeto devolve um desses.

## Exceções

| Python                                     | Ruby                       |
|--------------------------------------------|----------------------------|
| <code>try</code>                           | <code>begin</code>         |
| <code>except</code>                        | <code>rescue</code>        |
| <code>finally</code>                       | <code>ensure</code>        |
| base para capturar: <code>Exception</code> | <code>StandardError</code> |

```
def call
  arriscado
rescue StandardError => e
  Rails.error.report(e)
  nil
end
```

Dentro de um método o `begin` é implícito — dá para escrever só o `rescue` no fim. Criando o seu tipo de erro:

```
class InvalidToken < StandardError; end

raise InvalidToken if token_ruim?
```

Herde sempre de `StandardError`, nunca de `Exception`: capturar `Exception` pega até `Ctrl+C`.

## Módulos e mixins

Ruby não tem herança múltipla. Tem **módulo**: um saco de métodos que você inclui numa classe.

```
class User < ApplicationRecord
  include EmailVerifiable
  include PasswordResetable
end
```

Vamos escrever esses dois módulos na Aula 2. O nome deles no mundo Rails é **concern**.

## Dependências

| Python                              | Ruby                                       |
|-------------------------------------|--------------------------------------------|
| <code>pip install</code>            | <code>gem install</code>                   |
| <code>requirements.txt</code>       | <code>Gemfile</code>                       |
| <code>venv</code> por projeto       | <code>bundler</code> resolve tudo          |
| <code>pip freeze</code> para travar | <code>Gemfile.lock</code> , gerado sozinho |

O `Gemfile.lock` é o `requirements.txt` travado — só que automático e **sempre versionado**. Ele é a garantia de que a sua máquina e o servidor rodam exatamente as mesmas versões.

## Prática rápida

```
irb
```

```
3.class          # Integer
"oi".class       # String
:oi.class        # Symbol
nil.class        # NilClass

5.times { |i| puts i }

usuario = { nome: "Ana", role: :admin }
usuario[:nome]

[1, 2, 3].map { |n| n * n }
```

## 6. Rails

Framework web em Ruby, criado por **David Heinemeier Hansson** em 2004 — extraído do Basecamp, um produto real. Traz tudo junto: rotas, banco, e-mail, filas, testes, deploy. Estamos na versão 8.

### As duas leis

**Convenção sobre configuração.** Você não configura o óbvio, você segue o combinado. Classe `User`? Então a tabela é `users`, o arquivo é `user.rb`, a chave primária é `id`. Ninguém precisa dizer. Seguindo a convenção você escreve quase nada; brigando com ela, o dobro.

**DRY** — *Don't Repeat Yourself*. Cada regra existe num lugar só.

E uma atitude: **omakase**, "deixa comigo, chef". O Rails já escolheu as ferramentas por você. Você pode trocar, mas só troque quando tiver um motivo real. Isso é liberdade: sobra tempo pro problema de verdade.

## Rails × Django

| Django                                             | Rails                                                    |
|----------------------------------------------------|----------------------------------------------------------|
| <code>models.py</code>                             | <code>app/models/</code> , um arquivo por classe         |
| <code>makemigrations</code> / <code>migrate</code> | <code>rails g migration</code> / <code>db:migrate</code> |
| <code>urls.py</code>                               | <code>config/routes.rb</code>                            |
| <code>views.py</code>                              | <code>app/controllers/</code>                            |
| Django ORM                                         | ActiveRecord                                             |
| <code>manage.py</code>                             | <code>bin/rails</code>                                   |

Quem vem de Flask ou FastAPI vai sentir mais diferença: lá você monta cada peça, aqui elas já vêm encaixadas.

## MVC

- **Model** — os dados e as regras. Conversa com o banco.
- **View** — a tela. Na nossa API, é o JSON.
- **Controller** — recebe a requisição e decide o que fazer.
- **Rota** — o mapa: este endereço vai para aquele controller.

O caminho é sempre: **rota** → **controller** → **model** → **resposta**.

## Zeitwerk: o nome diz o caminho

Você nunca escreve `require` no Rails. Por quê?

O **Zeitwerk** carrega a classe no instante em que você a menciona, e descobre o arquivo pelo **nome da classe**:

```
Api::V1::StatusController → app/controllers/api/v1/status_controller.rb
```

Errou o caminho, a classe simplesmente não existe. Não é questão de estilo — é o mecanismo que faz "convenção sobre configuração" funcionar de verdade.

## Os três ambientes

| Ambiente                 | Onde        | Como se comporta                                                |
|--------------------------|-------------|-----------------------------------------------------------------|
| <code>development</code> | sua máquina | recarrega o código a cada requisição, log verboso, erro na tela |
| <code>test</code>        | os testes   | banco separado, limpo a cada teste                              |
| <code>production</code>  | o servidor  | código congelado, log enxuto, erro genérico                     |

Cada um tem um arquivo em `config/environments/`. `Rails.env` diz onde você está.

É assim que o e-mail abre no navegador em desenvolvimento e sai por SMTP em produção — mesmo código, ambientes diferentes.

## 7. Mão na massa

### Criar o projeto

```
rails new automic_auth_api --api -d postgresql \
  --skip-action-mailbox --skip-action-text --skip-active-storage \
  --skip-jbuilder --skip-action-cable
cd automic_auth_api
```

- `--api` — sem HTML, sem CSS, sem sessão de navegador. Só JSON.
- `-d postgresql` — o mesmo banco que usamos em produção.
- Os `--skip` tiram peças que não vamos usar. Cada peça a menos é uma peça a menos para dar problema.

### Subir o banco

Crie o `compose.yaml` (ou copie o deste repositório) e rode:

```
docker compose up -d
docker compose ps          # tem que aparecer "healthy"
```

Se der `address already in use`, você já tem um Postgres na máquina ocupando a 5432. Rode `DB_PORT=5433 docker compose up -d` e exporte `DB_PORT=5433` no shell.

### Preparar e subir a aplicação

```
bin/rails db:prepare
bin/rails server
```

Abra `http://localhost:3000/up`. Verde é a aplicação de pé.

## O tour das pastas

| Pasta                | O que tem                                                               |
|----------------------|-------------------------------------------------------------------------|
| <code>app/</code>    | O seu código. É onde você passa 90% do tempo.                           |
| <code>config/</code> | Rotas, banco, ambientes, credenciais                                    |
| <code>db/</code>     | Migrations e o retrato atual do banco ( <code>schema.rb</code> )        |
| <code>test/</code>   | Os testes                                                               |
| <code>bin/</code>    | Os comandos: <code>rails</code> , <code>setup</code> , <code>dev</code> |
| <code>Gemfile</code> | As dependências                                                         |

## Os comandos do dia a dia

```
bin/rails server      # sobe a aplicação
bin/rails console     # um irb com o seu projeto carregado dentro
bin/rails routes      # todas as rotas que existem
bin/rails generate    # cria arquivos seguindo a convenção
bin/rails db:migrate  # aplica as mudanças de banco
bin/rails test        # roda os testes
```

O **console** é a ferramenta mais subestimada do Rails: dá para criar usuário, rodar query e testar método sem escrever um arquivo.

## 8. A primeira rota

`config/routes.rb`:

```
Rails.application.routes.draw do
  get "up" => "rails/health#show", as: :rails_health_check

  namespace :api do
    namespace :v1 do
      get "status", to: "status#show"
    end
  end
end
```

`app/controllers/api/v1/status_controller.rb`:

```

module Api
  module V1
    class StatusController < ApplicationController
      def show
        render json: {
          status: "ok",
          service: "atomic-auth-api",
          environment: Rails.env
        }
      end
    end
  end
end

```

Repare na convenção: `namespace :api` + `namespace :v1` + `status#show` implica exatamente o arquivo `app/controllers/api/v1/status_controller.rb`, classe `Api::V1::StatusController`, método `show`. Ninguém configurou isso.

## Teste

`test/controllers/api/v1/status_controller_test.rb`:

```

require "test_helper"

class Api::V1::StatusControllerTest < ActionDispatch::IntegrationTest
  test "responde ok" do
    get "/api/v1/status"

    assert_response :ok
    assert_equal "ok", response.parsed_body["status"]
  end
end

```

```
bin/rails test
```

## Exercício da aula

**Onde você trabalha:** no **seu** projeto, que você cria no passo 1. O repositório da capacitação é o **gabarito** — abra para consultar, não para escrever.

## 1. Crie o seu projeto

```
cd ~
rails new automic_auth_api --api -d postgresql \
  --skip-action-mailbox --skip-action-text --skip-active-storage \
  --skip-jbuilder --skip-action-cable
cd automic_auth_api
```

## 2. Publique no seu GitHub

Não é opcional: na Aula 4, o deploy automático precisa de um repositório **seu**.

```
git add -A && git commit -m "Projeto inicial"
gh repo create automic_auth_api --private --source=. --push
```

Sem o `gh` instalado, crie o repositório pelo site e depois:

```
git remote add origin git@github.com:SEU-USUARIO/automic_auth_api.git
git push -u origin main
```

## 3. Suba o banco

Crie um arquivo `compose.yaml` na raiz do projeto:

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: automic
      POSTGRES_PASSWORD: automic
      POSTGRES_DB: automic_auth_api_development
    ports:
      - "${DB_PORT:-5432}:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U automic"]
      interval: 5s
      retries: 10

volumes:
  pgdata:
```

E em `config/database.yml`, dentro do bloco `default: &default`, acrescente:

```
host: <%= ENV.fetch("DB_HOST", "localhost") %>
port: <%= ENV.fetch("DB_PORT", 5432) %>
username: <%= ENV.fetch("DB_USER", "automic") %>
password: <%= ENV.fetch("DB_PASSWORD", "automic") %>
```

```
docker compose up -d
docker compose ps      # tem que aparecer "healthy"
```

address already in use? Você já tem um Postgres na 5432. Rode `DB_PORT=5433 docker compose up -d` e `export DB_PORT=5433` no shell.

## 4. Suba a aplicação

```
bin/rails db:prepare
bin/rails server
```

Abra `http://localhost:3000/up`. Verde é a aplicação de pé.

## 5. Escreva a rota

Em `config/routes.rb`, dentro do `Rails.application.routes.draw do`:

```
namespace :api do
  namespace :v1 do
    get "status", to: "status#show"
  end
end
```

Crie o arquivo `app/controllers/api/v1/status_controller.rb` — o caminho tem que ser **exatamente esse**, é o Zeitwerk:

```
module Api
  module V1
    class StatusController < ApplicationController
      def show
        render json: {
          status: "ok",
          service: "automic-auth-api",
          environment: Rails.env
        }
      end
    end
  end
end
```

## 6. Confira

```
bin/rails routes -g api      # a sua rota tem que aparecer
curl localhost:3000/api/v1/status
```

Depois monte a mesma requisição no Insomnia e confira o `200`.

## 7. Escreva o teste

Crie `test/controllers/api/v1/status_controller_test.rb`:

```
require "test_helper"

class Api::V1::StatusControllerTest < ActionDispatch::IntegrationTest
  test "responde ok" do
    get "/api/v1/status"

    assert_response :ok
    assert_equal "ok", response.parsed_body["status"]
  end
end
```

```
bin/rails test
```

## 8. Commit

```
git add -A && git commit -m "Rota de status" && git push
```

**Bônus:** faça a rota devolver também `RUBY_VERSION` e `Rails.version`.

**Travou?** Abra o mesmo arquivo no gabarito, entenda o que está diferente, e conserte o seu:

```
cd ~/capacitacao-gabarito && git checkout aula-01
cat app/controllers/api/v1/status_controller.rb
```

## Recapitulando

- Back-end é a cozinha: regra, dado e permissão.
- HTTP é o idioma; verbo, status e JSON são o vocabulário.
- HTTP não tem memória — segure essa ponta até a Aula 3.
- Ruby é Python com outra sintaxe e mais açúcar.
- Rails decide o óbvio por você. Siga a convenção.

**Na próxima:** o banco de dados entra em cena, e a API ganha um `User` com senha de verdade.

# Aula 2 — Banco de dados, ActiveRecord e o model `User`

---

**Duração:** ~3h · **Você sai daqui com:** um `User` com validações e senha hashada, testado. **Gabarito:**

```
cd ~/capacitacao-gabarito && git checkout aula-02
```

---

## 1. Por que Postgres, e não SQLite

---

SQLite é um arquivo. Funciona muito bem para um app de celular ou um script. Para um servidor com várias requisições ao mesmo tempo, ele trava — só uma escrita por vez no banco inteiro.

Postgres é um servidor: aguenta concorrência, tem tipos ricos (`jsonb`, arrays, intervalos de tempo), índices parciais e transações de verdade. É o que o `seem-backend` usa em produção, então é o que usamos aqui — **desenvolver no mesmo banco da produção evita a categoria inteira de bug que só aparece no deploy.**

---

### Transação

Um bloco de operações que acontece **inteiro ou não acontece**. Deu erro no meio, o banco desfaz tudo — isso se chama *rollback*.

O exemplo clássico é transferir dinheiro: debitar de um e creditar no outro têm que ser a mesma operação. Debitar sozinho é dinheiro que sumiu.

```
User.transaction do
  user.save!
  user.invalidate_sessions!
  user.clear_password_reset_code!
end
```

Repare no `!`: dentro de uma transação você **quer** que estoure. `save` devolve `false` em silêncio e a transação seguiria feliz, gravando metade.

Na Aula 3, trocar a senha vai precisar exatamente disso.

---

## 2. ActiveRecord

---

É o ORM do Rails: cada classe é uma tabela, cada objeto é uma linha.

| Django ORM                                              | ActiveRecord                               |
|---------------------------------------------------------|--------------------------------------------|
| <code>User.objects.filter(role="admin")</code>          | <code>User.where(role: "admin")</code>     |
| <code>User.objects.get(id=1)</code>                     | <code>User.find(1)</code>                  |
| <code>User.objects.all().order_by("-created_at")</code> | <code>User.order(created_at: :desc)</code> |
| <code>User.objects.count()</code>                       | <code>User.count</code>                    |
| <code>u.save()</code>                                   | <code>u.save</code>                        |

A diferença de filosofia: no Django você declara os campos na classe. **No Rails a classe não declara nada** — ela lê as colunas do banco em tempo de execução. A fonte da verdade é a migration.

```
class User < ApplicationRecord
end
```

Isso já tem `name`, `email`, `id`, `created_at` — tudo que existir na tabela `users`.

## ActiveSupport: os métodos que o Rails inventou

O Rails adiciona métodos às classes do próprio Ruby. Isso é a gem `activesupport`, não Ruby puro — fora do Rails esses métodos somem.

```
15.minutes
1.day.ago
2.weeks.from_now
Time.current      # respeita o fuso configurado na app; Time.now não
```

E o par que você vai usar mais que qualquer outro:

```
nil.blank?      # true
"".blank?       # true
"  ".blank?     # true  ← só espaço também conta como vazio
[].blank?       # true
0.blank?        # false  ← zero NÃO é vazio

"oi".present?  # true  ← present? é o contrário de blank?
```

**Pegadinha para quem vem de Python:** em Ruby puro, só `nil` e `false` são falsos. `if 0` executa. `if ""` executa. É justamente por isso que `blank?` existe.

### 3. Migrations

Uma migration é uma **mudança no banco, versionada em código**. Ela roda uma vez, em ordem, em todas as máquinas: a sua, a do colega e o servidor.

```
bin/rails generate migration CreateUsers
```

Isso cria `db/migrate/20260803142949_create_users.rb`. O número é a data e hora — é ele que define a ordem.

```
class CreateUsers < ActiveRecord::Migration[8.1]
  def change
    create_table :users do |t|
      t.string :name, null: false
      t.string :email, null: false
      t.string :password_digest, null: false
      t.string :course, null: false
      t.string :matricula, null: false
      t.datetime :terms_accepted_at, null: false

      t.timestamps
    end

    add_index :users, :email, unique: true
    add_index :users, :matricula, unique: true
  end
end
```

```
bin/rails db:migrate
```

Repare em duas coisas:

- `t.timestamps` cria `created_at` e `updated_at`, e o Rails mantém as duas sozinho.
- `add_index ... unique: true` é uma restrição no banco, não só validação no model. Isso importa: duas requisições simultâneas passam pela validação juntas (as duas consultam e as duas não acham ninguém), mas só uma vence o índice único. **Validação de model é para dar mensagem bonita; índice é para garantir.**

`schema.rb`

Depois de migrar, o Rails reescreve `db/schema.rb` — o retrato atual do banco. É ele que `db:prepare` usa para criar o banco de teste, e por isso **vai versionado**. Você nunca edita esse arquivo à mão.

#### Migrations são incrementais

Neste projeto a tabela `users` nasceu em três migrations, porque as funcionalidades chegaram em momentos diferentes:

```
20260803142949_create_users.rb          # o básico
20260803142950_add_email_verification_to_users.rb # confirmação de e-mail
20260803142951_add_password_reset_to_users.rb  # recuperação de senha
```

É assim que acontece na vida real. Nunca edite uma migration que já rodou em produção — crie outra.

## 4. Senha: o que nunca fazer

**Nunca guarde senha em texto.** Se o banco vazar — e bancos vazam — você entregou a senha de todos. E como as pessoas repetem senha, você entregou o e-mail e o banco delas junto.

### Hash não é criptografia

Criptografia tem volta (com a chave). **Hash não tem volta:** é um caminho só. Você não guarda a senha; guarda o resultado de passar a senha pela função. Na hora do login, passa de novo e compara os resultados.

### Por que bcrypt e não SHA-256

SHA-256 é rápido — **e isso é o problema.** Uma GPU testa bilhões de SHA-256 por segundo. bcrypt foi feito para ser **lento de propósito** e tem um fator de custo ajustável: quando o hardware melhora, você aumenta o custo.

bcrypt também gera um **salt** aleatório por senha. Sem salt, duas pessoas com a mesma senha teriam o mesmo hash, e uma tabela pronta (*rainbow table*) quebraria as duas de uma vez.

```
$2a$12$R9h/cIPz0gi.URNNX3kh20...
|  |  |  salt + hash
|  |  |  fator de custo (12 = 2^12 iterações)
|  |  |  versão do algoritmo
```

#### has\_secure\_password

```
# Gemfile
gem "bcrypt", "~> 3.1.7"
```

```
class User < ApplicationRecord
  has_secure_password validations: false
end
```

Isso dá ao model:

- `user.password = "..."` — recebe a senha em texto e grava o digest em `password_digest`
- `user.authenticate("...")` — devolve o usuário se bater, `false` se não

- `user.password_confirmation` — a confirmação, se você usar

A senha em texto **nunca toca o banco**. Ela vive na memória durante a requisição e some.

`validations: false` desliga as validações padrão porque a nossa política de senha é mais exigente. Ela mora em `PasswordPolicyValidator`.

## 5. Validações

```
validates :name, presence: true, length: { maximum: 120 }
validates :email, presence: true,
  uniqueness: { case_sensitive: false },
  format: { with: URI::MailTo::EMAIL_REGEXP }
validates :course, presence: true
validates :matricula, presence: true, uniqueness: true, format: { with: /\A\d{7}\z/ }
validates :terms_accepted_at, presence: true, on: :create
validate :password_meets_policy, if: -> { password.present? }
```

- `validates` (plural) usa um validador pronto. `validate` (singular) chama um método seu.
- `on: :create` só valida na criação — os termos são aceitos uma vez.
- `if:` recebe um lambda. Só valida a senha quando ela foi informada (na edição de perfil ela não é).

No console:

```
u = User.new(name: "Teste")
u.valid?           # false
u.errors.full_messages
```

### Regra de ouro: normalize antes de validar

`Ana@UFOP.br` e `ana@ufop.br` são o mesmo e-mail. `20.112-34` e `2011234` são a mesma matrícula.

```
def self.normalize_email(email)
  email.to_s.strip.downcase
end

def self.normalize_matricula(matricula)
  matricula.to_s.gsub(/\D/, "")
end
```

Sem isso o índice único não serve para nada: o banco acha que são valores diferentes.

## 6. Associações

Uma tabela nunca basta. Todo sistema real tem tabelas que se referem umas às outras: usuário tem muitos pedidos; palestra acontece numa sala; sala tem muitas palestras.

Vamos criar a segunda tabela do projeto: `login_events`, o histórico de acessos da conta. É o que o seu banco usa para mandar "novo acesso detectado" por e-mail.

### Quem guarda a chave

A relação um-para-muitos mora numa coluna só: a **chave estrangeira**. `login_events.user_id` aponta para `users.id`.

- quem **carrega** a coluna usa `belongs_to`;
- quem é **apontado** usa `has_many`.

Regra prática: a chave fica sempre no lado "muitos".

### A migration

```
create_table :login_events do |t|
  t.references :user, null: false, foreign_key: true
  t.string :client, null: false
  t.string :ip_address
  t.datetime :occurred_at, null: false
  t.timestamps
end

add_index :login_events, [ :user_id, :occurred_at ]
```

- `t.references :user` cria a coluna `user_id`, o índice **e** a chave estrangeira.
- `foreign_key: true` faz o **banco** recusar uma linha órfã — não é só validação de model.
- O índice composto tem `user_id` primeiro porque a consulta é sempre "os últimos logins **deste** usuário". Num índice composto, **a ordem das colunas importa**: primeiro o que filtra.

## Os dois lados

```
class User < ApplicationRecord
  has_many :login_events, dependent: :delete_all
end

class LoginEvent < ApplicationRecord
  belongs_to :user

  scope :recentes, -> { order(occurred_at: :desc) }
end
```

- `dependent: :delete_all` — apagar o usuário apaga o histórico junto. Sem isso sobram linhas apontando para um `id` que não existe mais.
- `belongs_to` já exige presença desde o Rails 5: `LoginEvent` sem `user` é inválido.
- `scope` é uma consulta com nome, e ela encadeia.

## Usando

```
# Criar PELA associação já preenche o user_id — não precisa passar:
user.login_events.create!(client: "mobile", occurred_at: Time.current)

user.login_events.count
user.login_events.recentes.limit(5)
evento.user.name # navega para o outro lado
```

## A armadilha N+1

```
users.each { |u| puts u.login_events.count }
```

Com 100 usuários isso são **101 consultas**: uma para buscar os usuários e cem para buscar o histórico de cada um. É o problema de performance número 1 de aplicação Rails.

A correção é uma palavra:

```
User.includes(:login_events).each { |u| puts u.login_events.count }
```

Duas consultas, sempre. O `seem-backend` usa a gem **Bullet**, que estoura um erro em desenvolvimento quando você escreve um N+1 sem perceber.

## 7. Concerns — o mixin da Aula 1, na prática

O `User` vai ganhar confirmação de e-mail e recuperação de senha. São dois assuntos que não conversam entre si. Jogar os dois no `user.rb` produz um arquivo de 800 linhas que ninguém abre com vontade.

Cada assunto vira um módulo em `app/models/concerns/`:

```
# app/models/concerns/email_verifiable.rb
module EmailVerifiable
  extend ActiveSupport::Concern

  CODE_TTL = 15.minutes
  RESEND_COOLDOWN = 1.minute

  included do
    scope :email_verified, -> { where.not(email_verified_at: nil) }
  end

  def email_verified?
    email_verified_at.present?
  end

  def issue_email_verification_code!
    code = format("%06d", SecureRandom.random_number(1_000_000))

    update!(
      email_verification_code_digest: BCrypt::Password.create(code),
      email_verification_expires_at: CODE_TTL.from_now,
      email_verification_sent_at: Time.current
    )

    code
  end
end
```

```
class User < ApplicationRecord
  include EmailVerifiable
  include PasswordResetable
end
```

- `extend ActiveSupport::Concern` habilita o bloco `included do ... end`, onde vão coisas que só fazem sentido na classe (scopes, validações, associações).
- O código completo dos dois concerns está em `app/models/concerns/`.

## Três decisões dentro do concern, e o porquê de cada uma

- 1. O código de 6 dígitos também vira digest bcrypt.** Ele é uma senha temporária. Quem lê o banco não pode confirmar a conta de ninguém.
- 2. O código em texto só existe no retorno do método.** `issue_email_verification_code!` devolve o código para o service mandar por e-mail, e depois ele some. Não fica em coluna, nem em log.
- 3. `email_verified_at` é data, não booleano.** Um booleano responde "sim". Uma data responde "sim, em 12 de agosto às 14h" — e isso você vai querer saber quando alguém abrir um chamado.

## 8. O console

```
bin/rails console
```

```
u = User.new(
  name: "Ana Souza",
  email: "ana@aluno.ufop.edu.br",
  password: "Automic@2026",
  course: "Engenharia de Controle e Automação",
  matricula: "2011234",
  terms_accepted_at: Time.current
)

u.valid?
u.save
u.password_digest      # o hash, não a senha
u.authenticate("Automic@2026") # devolve o user
u.authenticate("errada")      # false

User.count
User.where("email LIKE ?", "%ufop%")
User.find_by(email: "ana@aluno.ufop.edu.br")

codigo = u.issue_email_verification_code!
u.verify_email_code!(codigo)
u.email_verified?
```

`bin/rails console --sandbox` desfaz tudo ao sair. Bom para experimentar sem sujar o banco.

## 9. Seeds e fixtures

**Seeds** povoam o banco de desenvolvimento:

```
bin/rails db:seed
```

**Fixtures** são os dados dos testes, em `test/fixtures/users.yml`. O Rails carrega antes de cada teste e limpa depois — todo teste começa do mesmo estado.

```
ana:
  name: Ana Souza
  email: ana@aluno.ufop.edu.br
  password_digest: <%= BCrypt::Password.create("Automic@2026") %>
  email_verified_at: <%= 1.day.ago.to_fs(:db) %>
```

No teste, `users(:ana)` devolve esse registro.

## 10. Testando o model

```
test "guarda o digest e nunca a senha em texto" do
  user = build_user
  user.save!

  assert_not_equal "Automic@2026", user.password_digest
  assert user.authenticate("Automic@2026")
  assert_not user.authenticate("outra-senha")
end
```

```
bin/rails test
bin/rails test test/models/user_test.rb          # só um arquivo
bin/rails test test/models/user_test.rb:42       # só um teste
```

O Rails vem com **Minitest**. Se você conhece `pytest`, a ideia é a mesma; a sintaxe é `assert_*`.

## 11. Uma sutileza de segurança: `authenticate_by_email`

```
DUMMY_PASSWORD_DIGEST = BCrypt::Password.create("automic-timing-safe-dummy").freeze

def self.authenticate_by_email(email, password)
  user = find_by(email: normalize_email(email))
  digest = user&.password_digest || DUMMY_PASSWORD_DIGEST
  autenticado = BCrypt::Password.new(digest).is_password?(password.to_s)

  user if autenticado
end
```

Por que esse `DUMMY_PASSWORD_DIGEST`?

O jeito ingênuo seria `return nil unless user`. Só que bcrypt é lento **de propósito**. Se o e-mail não existe, a resposta volta em 1ms; se existe e a senha está errada, volta em 100ms. Cronometrando as respostas, dá para descobrir quem tem conta no sistema — é um **ataque de temporização**.

Rodando o bcrypt sempre, com um digest descartável quando não há usuário, as duas respostas custam o mesmo. Vamos usar esse método na Aula 3.

## Exercício da aula

1. Crie as três migrations e rode `bin/rails db:migrate`.
2. Abra `db/schema.rb` e confira que as colunas estão lá.
3. Escreva o `User` com `has_secure_password` e as validações.

4. Crie os dois concerns e inclua no `User`.
5. No console: crie um usuário, veja o `password_digest`, teste `authenticate`.
6. Emita um código de verificação e confirme com ele. Depois tente confirmar de novo — tem que recusar.
7. Escreva os testes e rode `bin/rails test`.
8. Crie a migration de `login_events`, o model com `belongs_to` e o `has_many` no `User`.
9. No console: `users(:ana).login_events.create!(...)`, depois `user.login_events.recentes`.
10. Apague um usuário que tenha eventos e confira que o histórico foi junto.

**Bônus:** escreva o N+1 de propósito (`User.all.each { |u| puts u.login_events.count }`), olhe o log para contar as consultas, e conserte com `includes`.

Travou? Compare com o gabarito: `cd ~/capacitacao-gabarito && git checkout aula-02`.

---

## Recapitulando

---

- Migration é a fonte da verdade do banco. O model não declara colunas.
- Índice único é garantia; validação é mensagem bonita. Você quer os dois.
- Senha vira hash bcrypt, com salt, e nunca volta.
- Concern é o mixin do Rails: um assunto por arquivo.
- Normalize antes de validar, ou o índice único não serve para nada.
- A chave estrangeira fica no lado "muitos": `belongs_to` carrega, `has_many` é apontado.
- Transação é tudo ou nada. Dentro dela, use os métodos com `!`.
- `includes` resolve N+1, e N+1 é o problema de performance mais comum em Rails.

**Na próxima:** as rotas. O usuário vai conseguir se cadastrar, entrar, sair e recuperar a senha.

# Aula 3 — As rotas de autenticação

**Duração:** ~3h · **Você sai daqui com:** cadastro, ativação, login, logout e recuperação de senha.

**Gabarito:** `cd ~/capacitacao-gabarito && git checkout aula-03`

## 1. O caminho de uma requisição

```
requisição → rota → controller → service → model → banco
                        ↓
                    serializer → JSON
```

Nesta aula todas essas peças aparecem. A regra que organiza tudo:

**Controller recebe requisição e devolve resposta. Regra de negócio não mora nele.**

## 2. A arquitetura de pastas

```
app/controllers/
├─ application_controller.rb
├─ concerns/api/v1/
│  ├─ authentication.rb      # quem está chamando?
│  └─ error_rendering.rb    # um formato de erro só
└─ api/v1/
   ├─ base_controller.rb    # pai de todos: erros, auth, strong params
   ├─ registrations_controller.rb
   ├─ email_verifications_controller.rb
   ├─ sessions_controller.rb
   ├─ password_resets_controller.rb
   └─ me_controller.rb

app/services/                # os casos de uso
├─ auth/
└─ users/

app/serializers/             # o que vai para o JSON
```

### Por que service object

O `Users::Register` faz seis coisas: valida campos obrigatórios, aplica a política de senha, confere a confirmação, normaliza os dados, cria o usuário e dispara o e-mail. Dentro do controller isso vira um método de 60 linhas que só dá para testar subindo uma requisição HTTP inteira.

Fora dele:

```
result = Users::Register.call(name: "...", email: "...", ...)
result.success? # true/false
result.user
result.errors # [{ field: "password", message: "..."}]
```

Testável direto, reaproveitável (uma task de importação chama o mesmo service) e o controller volta a ter cinco linhas. O padrão sempre igual: **um call de classe, um Result**.

## Por que serializer

```
render json: user
```

Isso manda o objeto inteiro: `password_digest`, `email_verification_code_digest`, `password_reset_code_digest`. Vazamento em uma linha.

O serializer é a lista de convidados: só entra quem está escrito lá.

```
class UserSerializer
  def self.as_json(user)
    { id: user.id, name: user.name, email: user.email, ... }
  end
end
```

## Um formato de erro só

```
{
  "error": {
    "code": "validation_failed",
    "message": "Falha na validação",
    "details": [{ "field": "password", "message": "deve incluir pelo menos um dígito" }]
  }
}
```

- **code** é estável — o cliente decide o que fazer com base nele.
- **message** é para o usuário ler. Pode mudar, pode ser traduzida.
- **details** diz qual campo falhou, para o app pintar o input de vermelho.

Uma API que devolve erro de três jeitos diferentes força o cliente a tratar os três.

## `before_action`: o filtro

```
class MeController < BaseController
  before_action :authenticate_user!

  def show
    render_user(current_user) # só roda se passou pelo filtro
  end
end
```

O filtro roda **antes** da action. Se ele renderizar alguma coisa — o `401` — a action nem chega a ser chamada. `only:` e `except:` limitam a quais actions ele se aplica:

```
before_action :authenticate_user!, only: :destroy
```

## A pilha de middleware

A requisição não cai direto no controller. Ela atravessa uma pilha de camadas — o **middleware**. Cada camada pode ler, alterar, ou responder e cortar o caminho ali mesmo.

```
requisição
  ↓
[ Rack::Cors ]           ← libera (ou não) a origem
[ ActionDispatch::Cookies ]
[ ... ]
  ↓
rota → before_action → controller → service → model
  ↓
serializer → JSON → resposta (voltando pela mesma pilha)
```

```
bin/rails middleware # lista a pilha inteira do seu app
```

O modo `--api` remove várias camadas. Trouxemos os cookies de volta na mão, em `config/application.rb`:

```
config.middleware.use ActionDispatch::Cookies
```

## Strong parameters

```
def registration_params
  expect_root_params(:name, :email, :password, :confirm_password,
                    :course, :matricula, :check_terms_use)
end
```

`params.expect` (Rails 8) **exige** essas chaves e **recusa o resto**. Sem isso alguém manda `"role": "admin"` no cadastro e vira admin. O nome disso é **mass assignment**, e já derrubou sistema grande.

### 3. Rota 1 — Cadastro

POST /api/v1/registrations

```
{
  "name": "Diego Reis",
  "email": "diego@aluno.ufop.edu.br",
  "password": "Automic@2026",
  "confirm_password": "Automic@2026",
  "course": "Engenharia de Minas",
  "matricula": "2013333",
  "check_terms_use": true
}
```

→ `201 Created` com o usuário e um e-mail de ativação a caminho.

#### A decisão que não é óbvia

```
REGISTRATION_FAILED_MESSAGE =
  "Não foi possível concluir o cadastro. Verifique os dados e tente novamente."
```

Quando o e-mail já existe, a resposta **não** diz "este e-mail já está cadastrado". Se dissesse, o cadastro viraria um verificador: eu testo mil e-mails e descubro quem tem conta no sistema. Chama-se **enumeração de usuários**, e vai voltar duas vezes nesta aula.

O custo é real: o usuário legítimo que esqueceu que já tem conta fica sem uma mensagem clara. A saída é o texto convidar a usar "esqueci minha senha".

### 4. E-mail: o mailer

Um mailer é um controller cujas views são e-mails.

```
class UserMailer < ApplicationMailer
  def email_verification(user, code)
    @user = user
    @code = code
    mail(to: @user.email, subject: "Ative sua conta")
  end
end
```

O template fica em `app/views/user_mailer/email_verification.text.erb`.

**Em desenvolvimento não existe SMTP.** O `letter_opener` abre o e-mail no navegador:

```
config.action_mailer.delivery_method = :letter_opener
```

`deliver_now` e não `deliver_later`

```
UserMailer.email_verification(user, code).deliver_now
```

O jeito "certo" de livro seria `deliver_later`, jogando numa fila. Aqui não, e o motivo é o código: numa fila, o código de 6 dígitos ficaria **gravado em texto** na tabela de jobs.

O preço é a requisição esperar o SMTP. Por isso o service tem `rescue`: falha de envio não pode virar 500.

```
rescue StandardError => e
  Rails.error.report(e, handled: true, source: "SendEmailVerification")
  Result.new(success?: false, error_code: "delivery_failed", ...)
end
```

## 5. Ativação da conta

```
POST /api/v1/email_verifications/confirm { email, code }
POST /api/v1/email_verifications/resend  { email }
```

O `resend` **sempre** responde 200 com a mesma mensagem — conta inexistente, já verificada ou em cooldown são indistinguíveis. É a mesma regra do cadastro.

## 6. O problema: HTTP não tem memória

Lembra da Aula 1? Cada requisição começa do zero. Então como o servidor sabe que você já entrou?

Duas famílias de resposta:

| Sessão no servidor                                  | Token                                                 |
|-----------------------------------------------------|-------------------------------------------------------|
| O servidor guarda a sessão e manda um id no cookie  | O servidor manda um crachá assinado e não guarda nada |
| Toda requisição consulta o armazenamento de sessão  | A assinatura basta: só validar                        |
| Logout é apagar a sessão — imediato                 | Logout é o problema difícil (já chegamos lá)          |
| Escalar exige sessão compartilhada entre servidores | Qualquer servidor valida sozinho                      |

Escolhemos **token**, porque o cliente principal é um app nativo — que não tem cookie e roda em milhares de celulares.

## JWT

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJIsInZlciI6MH0 . 4pQ8L_5f...  
cabeçalho          payload          assinatura
```

Três partes em **Base64**, separadas por ponto.

**Base64 não é segredo.** É uma forma de escrever bytes usando só letras e números, para caber num cabeçalho HTTP — que é texto. Não tem chave, não tem criptografia, qualquer um desfaz:

```
bash echo eyJzdWIiOiJ9 | base64 -d # {"sub":2}
```

Aquele monte de caractere embaralhado do JWT é isso: texto disfarçado.

**O payload é público.** Cole um token em [jwt.io](https://jwt.io) e você lê tudo. A assinatura garante que ninguém **alterou** o conteúdo, não que ninguém **leu**. Nunca ponha no payload nada que não possa ser lido — nem CPF, nem e-mail, nem permissão sensível.

O nosso payload:

```
{  
  sub: user.id,           # subject: de quem é o token  
  ver: user.token_version, # versão das sessões  
  jti: SecureRandom.uuid, # id único deste token  
  exp: 24.hours.from_now.to_i, # expiração  
  iat: Time.current.to_i    # emitido em  
}
```

## O detalhe que quebra tudo

```
JWT.decode(token, secret, true, { algorithm: "HS256" })  
#                ^^^^
```

Esse `true` é a validação da assinatura. Com `false`, o `JWT.decode` aceita **qualquer** token — e qualquer pessoa forja o próprio acesso trocando o `sub`. Já foi CVE em várias bibliotecas.

O teste que prova isso está em `me_controller_test.rb`:

```
test "recusa token assinado com outro segredo" do  
  forjado = JWT.encode({ sub: users(:ana).id, ... }, "segredo-do-atacante", "HS256")  
  get "/api/v1/me", headers: auth_headers(forjado)  
  assert_response :unauthorized  
end
```

## 7. Rota 2 — Login

```
POST /api/v1/sessions { email, password, client }
```

`client` é `"mobile"` ou `"web"`, e decide **como** o token é entregue:

| Cliente             | Transporte                                                    | Por quê                                                    |
|---------------------|---------------------------------------------------------------|------------------------------------------------------------|
| <code>mobile</code> | cabeçalho<br><code>Authorization: Bearer &lt;token&gt;</code> | App nativo não tem cookie                                  |
| <code>web</code>    | cookie assinado e <code>httpOnly</code>                       | JavaScript não lê o cookie, então um XSS não rouba o token |

### O que é um cookie

Um par `nome=valor` que o servidor manda no cabeçalho `Set-Cookie`. O navegador guarda e **reenvia sozinho**, em toda requisição àquele site. É a memória que o HTTP não tem, colada por fora.

Quem faz esse trabalho é o navegador — o app nativo não participa disso. Por isso o `client`.

```
cookies.signed[AUTH_COOKIE] = {  
  value: token,  
  httponly: true,           # JavaScript não enxerga  
  secure: Rails.env.production?, # só trafega em HTTPS  
  same_site: :lax          # mitiga CSRF  
}
```

`signed` significa que o Rails carimba o valor: alterou na mão, o Rails recusa.

### XSS

*Cross-Site Scripting*: o atacante consegue rodar **JavaScript dentro da sua página**. Como? Você exibiu texto de usuário sem escapar — alguém salvou `<script>...</script>` como nome e a sua página imprimiu cru.

Com JavaScript rodando ali, ele lê tudo que o JavaScript lê. É exatamente por isso que `httpOnly` existe: o token no cookie fica **fora do alcance** do JavaScript, e um XSS não o rouba.

### CSRF

*Cross-Site Request Forgery*. Lembra que o navegador reenvia o cookie sozinho?

Um site malicioso monta um formulário apontando para a sua API. A vítima clica; o navegador anexa o cookie dela; a sua API obedece, achando que foi ela quem pediu.

`same_site: :lax` manda o navegador **não enviar o cookie** quando a requisição parte de outro site.

API com token no cabeçalho não sofre de CSRF: ninguém anexa o `Authorization` por você. O problema é exclusivo de quem autentica por cookie — ou seja, do nosso `client: "web"`.

## 401 × 403

```
when "email_unverified"
  status: :forbidden      # 403: sabemos quem é você, mas ainda não pode
else
  status: :unauthorized   # 401: não sabemos quem é você
end
```

### A mesma mensagem para os dois erros

E-mail inexistente e senha errada devolvem `invalid_credentials`, o mesmo código e a mesma mensagem. Terceira vez que a enumeração aparece.

E lembra do `DUMMY_PASSWORD_DIGEST` da Aula 2? Ele fecha o buraco pelo lado do **tempo**: não adianta a mensagem ser igual se a resposta volta em 1ms quando o e-mail não existe e em 100ms quando existe.

---

## 8. Rota 3 — Logout, o problema difícil

Um JWT é válido até expirar. O servidor não guarda sessão. **Então "sair" não existe** — o token continua funcionando por até 24h.

Três respostas possíveis, e nós usamos duas:

### a) Apagar no cliente

O app joga o token fora. Resolve o caso normal e **não resolve nada** se alguém copiou o token.

## b) Denylist por `jti` — revoga aquele token

```
module Auth::TokenDenylist
  def revoke(payload)
    jti = payload[:jti]
    ttl = payload[:exp].to_i - Time.current.to_i
    return false if jti.blank? || ttl <= 0

    Rails.cache.write(key(jti), true, expires_in: ttl.seconds)
  end

  def revoked?(jti)
    Rails.cache.exist?(key(jti))
  end
end
```

A sacada está no **TTL**: cada entrada vive exatamente o tempo que faltava para o token expirar. Depois disso o token morre sozinho e a entrada não serve mais. **A lista se limpa sem varredura e sem lixo acumulado.**

Isso revoga **um** token. Logout no celular não derruba o notebook — que é o comportamento certo.

## c) `token_version` — derruba tudo de uma vez

```
def token_version_matches?(submitted_version)
  token_version == submitted_version.to_i
end

def invalidate_sessions!
  update!(token_version: token_version + 1)
end
```

O token carrega o `ver` de quando foi emitido. Incrementar a coluna no banco invalida **todos** os tokens do usuário, em todos os dispositivos. Usamos isso na troca de senha.

## A verificação completa

```
def user_from_token(token)
  payload = Auth::DecodeToken.call(token) # 1. a assinatura confere?
  return if Auth::TokenDenylist.revoked?(payload[:jti]) # 2. foi revogado?

  user = User.find_by(id: payload[:sub])
  user if user&.token_version_matches?(payload[:ver]) # 3. a versão bate?
rescue Auth::DecodeToken::InvalidToken
  nil
end
```

## 9. Rota 4 — Recuperação de senha

Dois passos, e o mais interessante do dia.

```
POST /api/v1/password_resets/request { email }
POST /api/v1/password_resets/confirm { email, code, password, confirm_password }
```

### `request` sempre responde 200

```
GENERIC_MESSAGE = "Se o e-mail estiver cadastrado, enviamos um código para redefinir sua senha."
```

**Sempre.** E-mail inexistente, não verificado, em cooldown, SMTP fora do ar — mesma resposta, mesmo status. Se respondesse 404 para e-mail inexistente, esta rota pública viraria um verificador de cadastro. É a quarta vez que o assunto aparece: **em fluxo de autenticação, respostas diferentes vazam informação.**

Duas guardas dentro:

- **Só conta verificada** recebe código. Senão dá para sequestrar um cadastro feito com o e-mail de outra pessoa antes de ela confirmar.
- **Cooldown de 1 minuto**, senão qualquer um usa o seu servidor de e-mail para incomodar terceiros.

### `confirm` e a transação

```
User.transaction do
  user.save!           # troca a senha
  user.invalidate_sessions! # derruba todas as sessões ativas
  user.clear_password_reset_code! # consome o código
end
```

As três juntas ou nenhuma. Sem a transação, uma falha no meio deixa a senha trocada com o código ainda valendo.

E a ordem das validações importa: a política de senha é conferida **antes** de consumir o código. Senão o usuário que digita a senha nova errada perde o código e precisa pedir outro.

`invalidate_sessions!` **é o ponto do fluxo inteiro.** Se alguém invadiu a conta, trocar a senha tem que expulsar o invasor. Sem essa linha, ele continua logado com o token antigo.

## 10. Testando

```
bin/rails test
```

O teste que resume a aula:

```
test "logout revoga o token apresentado" do
  token = login_as(users(:ana))

  get "/api/v1/me", headers: auth_headers(token)
  assert_response :ok

  delete "/api/v1/sessions", headers: auth_headers(token)

  get "/api/v1/me", headers: auth_headers(token)
  assert_response :unauthorized # sem a denylist, isto daria 200
end
```

**Pegadinha:** em teste o `Rails.cache` padrão é o `null_store` — não guarda nada. O teste acima passaria "de graça", provando nada. Por isso o `test_helper.rb` troca por um `MemoryStore`.

## As ferramentas de qualidade

```
bin/rubocop      # estilo – o black/ruff do Ruby
bin/brakeman     # análise estática de segurança
bundle exec bundler-audit check --update # gems com CVE conhecido
```

As três rodam no CI (Aula 4). O `brakeman` procura SQL injection, mass assignment, redirect aberto e mais uma dúzia de padrões.

## 11. CORS

```
origins(ENV.fetch("CORS_ORIGINS", "http://localhost:5173").split(","))
```

O navegador bloqueia, por padrão, uma página em `painel.exemplo.com` chamar `api.exemplo.com`. Esta config é a API dizendo quais origens aceita.

**O app nativo não passa por CORS** — isso é regra de navegador. Na prática, essa configuração existe por causa do painel web.

## 12. O fluxo inteiro, no terminal

```
API=localhost:3000/api/v1

# cadastro
curl -X POST $API/registrations -H 'Content-Type: application/json' -d '{
  "name":"Diego Reis","email":"diego@aluno.ufop.edu.br",
  "password":"Automic@2026","confirm_password":"Automic@2026",
  "course":"Engenharia de Minas","matricula":"2013333","check_terms_use":true}'

# login antes de confirmar → 403 email_unverified
curl -X POST $API/sessions -H 'Content-Type: application/json' \
  -d '{"email":"diego@aluno.ufop.edu.br","password":"Automic@2026","client":"mobile"}'

# pegue o código no e-mail que abriu no navegador, e confirme
curl -X POST $API/email_verifications/confirm -H 'Content-Type: application/json' \
  -d '{"email":"diego@aluno.ufop.edu.br","code":"123456"}'

# login (o token vem no cabeçalho Authorization da resposta)
curl -i -X POST $API/sessions -H 'Content-Type: application/json' \
  -d '{"email":"diego@aluno.ufop.edu.br","password":"Automic@2026","client":"mobile"}'

TOKEN=cole-o-token-aqui
curl $API/me -H "Authorization: Bearer $TOKEN"           # 200
curl -X DELETE $API/sessions -H "Authorization: Bearer $TOKEN"
curl $API/me -H "Authorization: Bearer $TOKEN"         # 401
```

## Exercício da aula

1. Monte as cinco rotas.
2. No **Insomnia**, crie uma coleção com uma requisição para cada uma.
3. Percorra o fluxo inteiro: cadastro → e-mail → confirmação → login → `/me` → logout → `/me` (401).
4. Peça recuperação de senha, troque a senha e confira que **o token antigo parou de funcionar**.
5. Cole o seu token em [jwt.io](https://jwt.io) e leia o payload. Ache o `sub`, o `jti` e o `exp`.
6. Rode `bin/rails test`, `bin/rubocop` e `bin/brakeman`.

**Bônus 1:** crie `PATCH /api/v1/me` para o usuário editar o próprio `name` — e só o `name`.

**Bônus 2:** descubra o que acontece se você trocar o `true` do `JWT.decode` por `false` e chamar `/me` com um token forjado. Depois desfaça.

Travou? Compare com o gabarito: `cd ~/capacitacao-gabarito && git checkout aula-03`.

## Recapitulando

---

- Controller recebe e responde; a regra mora no service.
- Serializer é a lista de convidados do JSON — sem ele o digest vaza.
- O payload do JWT é público. Assinado ≠ criptografado.
- Logout de verdade precisa de denylist por `jti`; o TTL faz a limpeza sozinho.
- `token_version` derruba tudo de uma vez, e é o que a troca de senha usa.
- Em autenticação, respostas diferentes vazam informação. Uma resposta só, sempre.

**Na próxima:** nada disso serve se está só na sua máquina. Vamos colocar no ar.

# Aula 4 — VPS, Docker, Kamal e deploy

**Duração:** ~3h · **Você sai daqui com:** a sua API no ar, em `https://seunome.capacita.<domínio>`, com deploy automático a cada push na `main`. **Gabarito:** `cd ~/capacitacao-gabarito && git checkout aula-04`

Este roteiro é a versão para a turma do guia de infraestrutura do `seem-backend`. As decisões são as mesmas; a máquina é menor.

## 1. O que é uma VPS

**VPS** = *Virtual Private Server*. Um computador virtual, dentro de um servidor físico de um provedor, que é seu: você tem acesso root, escolhe o sistema, instala o que quiser.

| Opção                             | O que você controla | O que você opera                               |
|-----------------------------------|---------------------|------------------------------------------------|
| Hospedagem compartilhada          | quase nada          | nada                                           |
| <b>PaaS</b> (Heroku, Render, Fly) | o app               | nada — mas paga mais e obedece as regras deles |
| <b>VPS</b>                        | tudo                | tudo: SO, atualizações, firewall, banco        |
| Kubernetes                        | tudo, em escala     | muito mais                                     |

### Por que VM com containers, e não Kubernetes

O `seem-backend` atende ~500 usuários num evento de uma semana. Kubernetes adicionaria custo e operação sem resolver um problema que existe. A escolha foi:

- menos peças para monitorar;
- deploy reproduzível com Docker e Kamal;
- banco e aplicação perto um do outro;
- recuperação simples numa VM substituta;
- **evoluir só quando uma métrica real pedir.**

Escolher a ferramenta grande antes do problema grande é a forma mais cara de errar.

**O que isso custa, e é honesto admitir:** uma VM é ponto único de falha. Volume Docker não é backup. O Postgres não é gerenciado — quem cuida dele é você.

## 2. Criar a VM na Azure

### Cotas antes de tudo

A assinatura Azure for Students não oferece todos os tamanhos em todas as regiões. O portal responde:

NotAvailableForSubscription

O caminho certo:

1. **Assinaturas** → **Azure for Students** → **Uso + cotas**
2. filtre por `Compute`
3. veja as cotas regionais
4. escolha uma região onde o tamanho aparece

Não insista num tamanho marcado como indisponível. Pedir aumento de cota não resolve quando a oferta não está habilitada para a assinatura.

### O assistente

**Máquinas virtuais** → **Criar** → **Máquina virtual do Azure**

| Campo                  | Valor                                                                 |
|------------------------|-----------------------------------------------------------------------|
| Grupo de recursos      | Novo: <code>rg-capacita-&lt;seunome&gt;</code>                        |
| Nome                   | <code>vm-capacita-&lt;seunome&gt;</code>                              |
| Região                 | uma com cota disponível                                               |
| Imagem                 | Ubuntu Server 24.04 LTS — x64 Gen2                                    |
| Tamanho                | <code>Standard_B1s</code> (1 vCPU, 1 GiB) ou maior, se houver crédito |
| Autenticação           | Chave pública SSH                                                     |
| Usuário                | <code>azureuser</code>                                                |
| Tipo de chave          | Ed25519                                                               |
| <b>Portas públicas</b> | <b>Nenhuma</b>                                                        |

**"Nenhuma" é a escolha mais importante da tela.** O assistente, se deixado, cria uma regra liberando SSH para a internet inteira. Bots varrem a porta 22 do IPv4 todo, o dia todo. Vamos abrir a 22 só para o seu IP.

Rede:

- VNet e sub-rede: aceite os padrões

- IP público: Standard
- NSG: avançado (é o firewall — vamos mexer nele)

Marque tudo com tags (`ambiente=capacitacao`, `dono=<seunome>`): é assim que você acha recurso órfão consumindo crédito depois.

## Antes: duas chaves, um par

**Criptografia assimétrica** usa duas chaves que se completam. A **pública** você espalha; a **privada** nunca sai da sua máquina. O que uma fecha, só a outra abre.

Daí saem duas coisas diferentes:

- **sigilo** — eu fecho com a *sua* pública, e só você abre;
- **assinatura** — eu fecho com a *minha* privada, e todo mundo confere que fui eu.

**É assim que o SSH funciona.** Você põe a sua chave pública no servidor (`~/.ssh/authorized_keys`). Ao conectar, o servidor manda um desafio; você responde assinando com a privada; o servidor confere com a pública que já tinha. **A senha nunca trafega — nem existe.**

E é por isso que perder a chave privada é perder o acesso à máquina.

Compare com o JWT da Aula 3: lá a assinatura usa uma chave **simétrica** (HS256) — a mesma chave assina e confere, porque quem assina e quem confere são o mesmo servidor.

## A chave privada

O portal oferece o download **uma vez**.

```
mkdir -p ~/.ssh
mv ~/Downloads/vm-capacita-seunome_key.pem ~/.ssh/azure-capacita
chmod 600 ~/.ssh/azure-capacita
```

`chmod 600` = só você lê. O SSH **recusa** usar uma chave com permissão frouxa.

Chave privada não vai por e-mail, não vai por WhatsApp, não vai para o Git. Nunca.

## Abrir o SSH só para você

No NSG da VM, **Regras de segurança de entrada** → **Adicionar**:

| Campo             | Valor            |
|-------------------|------------------|
| Origem            | Endereços IP     |
| IPs de origem     | SEU_IP/32        |
| Portas de destino | 22               |
| Protocolo         | TCP              |
| Prioridade        | 100              |
| Nome              | allow-ssh-meu-ip |

Descubra o seu IP:

```
curl -4 ifconfig.me
```

O `/32` significa "exatamente este endereço". Precisa também de:

| Porta | Para quê                             |
|-------|--------------------------------------|
| 80    | HTTP (o Cloudflare precisa alcançar) |
| 443   | HTTPS                                |

Internet de casa troca de IP. Quando o SSH parar de conectar do nada, é isso: atualize a regra.

## Primeiro acesso

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP_PUBLICO
```

## 3. Preparar o Ubuntu

### Atualizar

```
sudo apt update && sudo apt upgrade -y
test -f /var/run/reboot-required && sudo reboot
```

### Docker, do repositório oficial

O `apt install docker.io` do Ubuntu instala uma versão velha. Use o repositório da Docker:

```

sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

```

Libere o Docker para o `azureuser`:

```

sudo usermod -aG docker azureuser
exit

```

**Saia e entre de novo** — grupo só vale em sessão nova.

```

ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP
docker run --rm hello-world

```

## Root e permissões

`root` é o usuário que pode tudo — sem "tem certeza?". Você trabalha como `azureuser` e chama `sudo` quando precisa.

Todo arquivo tem dono, grupo e três permissões: ler, escrever, executar.

```

chmod 600 arquivo    # dono lê e escreve; mais ninguém vê nada
chmod 700 pasta      # dono entra; mais ninguém

```

O SSH **exige** `600` numa chave privada — com permissão mais frouxa ele recusa usar o arquivo.

E nunca rode a aplicação como root: o Dockerfile já cria um usuário `rails` justamente para isso.

## Swap

```

ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP 'sudo bash -s' < scripts/server-swap.sh

```

Numa VM de 1 GiB, Rails + Postgres + build estouram a RAM e o kernel mata o processo que estiver na frente (*OOM killer*), com uma mensagem que não explica nada. 2 GiB de swap resolvem.

## Endurecer o SSH

**Mantenha a sessão atual aberta** enquanto testa em outro terminal.

```
sudo tee /etc/ssh/sshd_config.d/99-hardening.conf >/dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
EOF

sudo sshd -t          # valida a sintaxe ANTES de recarregar
sudo systemctl reload ssh
```

Em outro terminal:

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP 'echo OK'
```

Só feche a primeira sessão depois que isso responder. Errar a config do SSH com uma sessão só aberta é o jeito clássico de perder acesso à máquina.

---

## 4. Docker

### Imagem × container

**Imagem** é a receita: o sistema, o Ruby, as gems, o seu código, congelados. **Container** é o bolo: uma instância rodando. Uma imagem, muitos containers.

### O Dockerfile do Rails

O Rails 8 gera um Dockerfile de produção pronto. Vale ler:

```
FROM ruby:3.4.9-slim AS base
# ...

FROM base AS build
RUN apt-get install -y build-essential libpq-dev ...
COPY Gemfile Gemfile.lock ./
RUN bundle install
COPY . .

FROM base
RUN groupadd --system --gid 1000 rails && useradd rails --uid 1000 --gid 1000
USER 1000:1000
COPY --from=build "${BUNDLE_PATH}" "${BUNDLE_PATH}"
COPY --from=build /rails /rails
```

Duas decisões importantes:

**Multi-stage.** O estágio `build` instala compilador e headers para compilar as gems nativas. A imagem final copia só o resultado. Compilador não vai para produção: menos peso e menos ferramenta à mão de quem invadir.

**Usuário não-root.** Se alguém escapar da aplicação, cai num usuário sem privilégio. É uma linha que muda o tamanho do estrago.

#### `.dockerignore`

Diz o que **não** entra na imagem: `.git`, `log/`, `tmp/`, `node_modules`. Sem ele a imagem fica gorda e — pior — o histórico do Git vai junto.

## Registry

O registry é o "GitHub das imagens". Usamos o **ghcr.io** (GitHub Container Registry): já vem com a conta e o Actions autentica sozinho com o `GITHUB_TOKEN`.

O fluxo: **Actions faz o build → empurra a imagem para o ghcr.io → a VM puxa e roda.**

A sua máquina nunca faz o build de produção.

---

## 5. Kamal

O Kamal (feito pela mesma turma do Rails) faz *zero-downtime deploy* com Docker, via SSH. Sem agente, sem painel: ele conecta na sua máquina e roda `docker`.

O que ele faz num `kamal deploy`:

1. builda a imagem e empurra para o registry
2. conecta na VM por SSH
3. puxa a imagem
4. sobe o container novo **ao lado** do antigo
5. espera o health check do novo passar
6. manda o tráfego para o novo e derruba o antigo

Falhou o health check? Ele não troca. O antigo continua servindo.

#### `config/deploy.yml`, por partes

```
service: automic-auth-api
image: <%= ENV.fetch("GHCR_USER") %>/automic-auth-api
```

O arquivo é ERB: dá para usar `ENV`. É por isso que este `deploy.yml` é **igual para todo mundo da turma** — o que muda vem das variables do Actions.

```
ssh:
  user: azureuser
run_directory: /home/azureuser/.kamal
```

Sem login root, então o Kamal precisa de um lugar para gravar lock e auditoria.

```
env:
  clear:
    SOLID_QUEUE_IN_PUMA: "true"
    WEB_CONCURRENCY: "1"
    DB_HOST: automic-auth-api-db
  secret:
    - RAILS_MASTER_KEY
    - JWT_SECRET
```

- `clear` vai como texto no comando `docker run`. Configuração, não segredo.
- `secret` vai por um arquivo de ambiente, com permissão restrita no servidor.

Colocar `JWT_SECRET` no `clear` seria o mesmo que publicá-lo: ele apareceria em `docker inspect` e no histórico do shell.

`DB_HOST: automic-auth-api-db` é o **nome do container** do banco. Containers na mesma rede Docker se enxergam por nome — não precisa de IP.

```
accessories:
  db:
    image: postgres:16
    directories:
      - data:/var/lib/postgresql/data
```

**Accessory** é um container que o Kamal gerencia mas não faz deploy junto com o app. O `directories` cria o volume: é ele que faz os dados sobreviverem a deploy e a rebuild.

`kamal deploy` **não** sobe accessories. Depois de mudar env de accessory: `kamal accessory reboot db`.

## As três camadas de segredo

```
GitHub Actions Secrets
  ↓ (viram variáveis de ambiente no runner)
.kamal/secrets
  ↓ (referencia as variáveis – nenhum valor aqui, por isso vai para o Git)
config/deploy.yml (env.secret)
  ↓ (o Kamal injeta no container)
ENV["JWT_SECRET"] no Rails
```

Nenhum valor real toca o repositório em nenhum ponto.

## 6. Segredos do Rails

`credentials` e `master.key`

```
bin/rails credentials:edit
```

O Rails abre um YAML no editor, e ao salvar grava `config/credentials.yml.enc` — criptografado, seguro no Git. A chave que decripta é `config/master.key`, que **nunca** vai para o Git (já está no `.gitignore`).

Em produção, `master.key` vira o secret `RAILS_MASTER_KEY`.

### Antes: o que é uma variável de ambiente

Um par `nome=valor` que o sistema operacional entrega ao processo quando ele sobe.

```
export JWT_SECRET=abc123
```

E o programa lê com `ENV["JWT_SECRET"]`.

**Por que não deixar o valor no código?** Porque o código vai para o Git. A ideia é: mesmo código, ambientes diferentes, valores diferentes. É um dos [12 fatores](#), e é como o Kamal injeta segredo no container.

### Credentials × ENV

|                    | Quando                                                               |
|--------------------|----------------------------------------------------------------------|
| <b>Credentials</b> | segredo estável, que é do app: chave de API de terceiro              |
| <b>ENV</b>         | o que muda por ambiente ou por máquina: senha do banco, host de SMTP |

Este projeto usa ENV para quase tudo, porque quem provisiona (Kamal) já sabe injetar.

Gere o `JWT_SECRET`:

```
bin/rails secret
```

## 7. GitHub Actions

### CI: o portão

`.github/workflows/ci.yml` roda em todo push e PR:

- `scan_ruby` — Brakeman (segurança estática) e bundler-audit (CVE em gems)

- `lint` — RuboCop
- `test` — `bin/rails test` contra um Postgres descartável

Se qualquer um falhar, o código não entra na `main`. Isso é o que faz "deploy automático" não ser "publicar bug automático".

## Deploy: o interessante

O ponto delicado é o SSH. A porta 22 está fechada para a internet. O runner do GitHub tem IP diferente a cada execução — não dá para liberar antes.

A sequência:

1. autentica na Azure (OIDC)
2. descobre o próprio IP público
3. cria uma regra no NSG liberando a 22 só para esse IP/32
4. faz o deploy
5. APAGA a regra — mesmo se o deploy falhar

O passo 5 tem `if: always()`. **Deixar a 22 aberta é o erro que transforma um deploy ruim num incidente de segurança.**

---

## 8. OIDC — por que não usar senha

O jeito comum seria criar um *client secret* na Azure e guardar como secret do GitHub. Problemas: ele é longo, vale por meses, e quem tiver acesso ao repositório tem acesso à sua Azure.

**OIDC** (OpenID Connect) inverte: o GitHub emite, a cada execução, um token de curta duração que diz "sou o workflow do repositório X, na branch Y". A Azure verifica e devolve um acesso temporário.

**Não existe segredo de longa duração em lugar nenhum.**

### Criando

**1. Registro de aplicativo** — Microsoft Entra ID → Registros de aplicativo → Novo registro:

- nome: `github-capacita-<seunome>`
- contas: somente este diretório
- URI de redirecionamento: vazia
- **não crie segredo do cliente**

Anote: Application (client) ID, Directory (tenant) ID, Subscription ID.

**2. Credencial federada** — no app criado, Certificados e segredos → Credenciais federadas → Adicionar → cenário **GitHub Actions**:

| Campo            | Valor                 |
|------------------|-----------------------|
| Organização      | seu usuário do GitHub |
| Repositório      | nome do repositório   |
| Tipo de entidade | Branch                |
| Branch           | <code>main</code>     |

O subject resultante:

```
repo:SEU-USUARIO/SEU-REPO:ref:refs/heads/main
```

Precisa bater **caractere por caractere** com o que o GitHub emite. Erro de maiúscula, de nome ou de branch dá `AADSTS700213` — e a mensagem não ajuda.

**3. Permissão mínima** — no **NSG** (não na assinatura), IAM → Adicionar atribuição de função → **Colaborador de Rede** → o app criado.

Escopo no NSG e não na assinatura: se a credencial vazar, o estrago se limita a regras de firewall daquela máquina.

**4. Chave SSH de deploy** — separada da sua chave pessoal:

```
ssh-keygen -t ed25519 -f ~/.ssh/capacita-github -C "github-actions" -N ""

ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP \
'umask 077; mkdir -p ~/.ssh; cat >> ~/.ssh/authorized_keys' < ~/.ssh/capacita-github.pub
```

Chave separada dá para revogar o acesso do CI sem trocar o seu.

## Cadastrando no GitHub

**Settings** → **Secrets and variables** → **Actions**

**Secrets:**

| Nome                                                                                | Conteúdo                                                    |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <code>AZURE_CLIENT_ID</code>                                                        | Application (client) ID                                     |
| <code>AZURE_TENANT_ID</code>                                                        | Directory (tenant) ID                                       |
| <code>AZURE_SUBSCRIPTION_ID</code>                                                  | Subscription ID                                             |
| <code>AZURE_SSH_PRIVATE_KEY</code>                                                  | conteúdo de <code>~/.ssh/capacita-github</code> (a privada) |
| <code>RAILS_MASTER_KEY</code>                                                       | conteúdo de <code>config/master.key</code>                  |
| <code>AUTOMIC_AUTH_API_DATABASE_PASSWORD</code>                                     | invente uma senha longa                                     |
| <code>JWT_SECRET</code>                                                             | saída de <code>bin/rails secret</code>                      |
| <code>SMTP_ADDRESS</code> / <code>SMTP_USERNAME</code> / <code>SMTP_PASSWORD</code> | do provedor de e-mail                                       |
| <code>KAMAL_PROXY_SSL_CERTIFICATE</code>                                            | certificado Origin CA                                       |
| <code>KAMAL_PROXY_SSL_PRIVATE_KEY</code>                                            | chave do Origin CA                                          |

**Variables** (não são segredo):

| Nome                              | Exemplo                                               |
|-----------------------------------|-------------------------------------------------------|
| <code>GHC_USER</code>             | seu usuário do GitHub                                 |
| <code>AZURE_VM_IP</code>          | <code>57.156.65.151</code>                            |
| <code>APP_HOST</code>             | <code>seunome.capacita.exemplo.tech</code>            |
| <code>AZURE_RESOURCE_GROUP</code> | <code>rg-capacita-seunome</code>                      |
| <code>AZURE_NSG_NAME</code>       | nome do NSG                                           |
| <code>CORS_ORIGINS</code>         | <code>http://localhost:5173</code>                    |
| <code>MAILER_FROM</code>          | <code>Capacitação &lt;noreply@exemplo.tech&gt;</code> |

Pelo CLI (não deixa o valor no histórico do shell):

```
gh secret set JWT_SECRET
gh secret set KAMAL_PROXY_SSL_CERTIFICATE < origin-ca.pem
gh variable set APP_HOST
```

## 9. Cloudflare e TLS

### HTTPS é HTTP dentro de um túnel

O HTTP da Aula 1 é **texto puro**: quem estiver no caminho — o roteador do café, o provedor — lê tudo, inclusive a senha. O **TLS** embrulha esse texto num túnel criptografado.

Mesmo protocolo, mesmos verbos, mesmos cabeçalhos, só que fechado. O "S" de HTTPS é isso, e nada além disso.

## O handshake, em três passos

1. O servidor apresenta o **certificado** dele.
2. O cliente confere se confia em **quem assinou** aquele certificado.
3. Os dois combinam uma chave temporária e conversam com ela.

O par de chaves assimétricas só serve para o passo 3. Depois disso a conversa usa criptografia **simétrica**, que é muito mais rápida.

## Certificado e autoridade

Um certificado diz: *"esta chave pública pertence a este domínio"* — e vem **assinado por uma Autoridade Certificadora (CA)**.

O seu navegador já nasce com uma lista de CAs em que confia. Confia na CA → confia em quem ela assinou. É uma cadeia.

- **Autoassinado** é você jurando que é você. O navegador não aceita.
- **Let's Encrypt** é uma CA pública e gratuita, em que os navegadores confiam.
- **Cloudflare Origin CA** *não* é pública — e é exatamente por isso que ela funciona aqui, como você vai ver.

## Proxy reverso

Um proxy comum fica na frente do **cliente**. Um proxy **reverso** fica na frente do **servidor**: recebe tudo na 443, termina o TLS, e repassa para a aplicação em HTTP interno.

Assim o Rails não precisa saber nada de certificado. Na nossa arquitetura há dois:

```
navegador → Cloudflare → kamal-proxy (na sua VM) → Rails
              (proxy 1)      (proxy 2)
```

É por isso que `config.assume_ssl = true`: ele avisa o Rails de que o "http" que chegou já veio de um "https" lá fora, e o Rails para de montar URLs erradas.

## DNS

No Cloudflare, no domínio da capacitação:

| Campo    | Valor                          |
|----------|--------------------------------|
| Tipo     | A                              |
| Nome     | seunome.capacita               |
| Conteúdo | o IP público da sua VM         |
| Proxy    | <b>Proxied</b> (nuvem laranja) |

**Proxied** significa que o tráfego passa pelo Cloudflare antes de chegar na sua VM. Você ganha cache, proteção contra DDoS e — o principal aqui — o IP real da sua máquina não aparece no DNS.

## Certificado Origin CA

SSL/TLS → **Origin Server** → Create Certificate. O Cloudflare gera o par e mostra **uma vez**. Copie os dois para os secrets `KAMAL_PROXY_SSL_CERTIFICATE` e `KAMAL_PROXY_SSL_PRIVATE_KEY`.

## Full (strict)

SSL/TLS → Overview → **Full (strict)**.

Os três modos:

| Modo                 | Cloudflare → sua VM                            |
|----------------------|------------------------------------------------|
| Flexible             | <b>em texto puro</b> — não use                 |
| Full                 | criptografado, mas aceita certificado inválido |
| <b>Full (strict)</b> | criptografado e o certificado é validado       |

Com `Flexible`, o cadeado aparece para o usuário e a senha dele trafega em claro no último trecho. É pior que não ter HTTPS, porque mente.

## Por que não Let's Encrypt

O Kamal sabe emitir Let's Encrypt sozinho. Aqui não dá:

- com o DNS **proxied**, o desafio HTTP-01 não chega ao seu servidor como o Let's Encrypt espera;
- daria para tirar o proxy durante a emissão, mas isso expõe o IP e vira ritual manual a cada renovação.

O Origin CA resolve: protege o trecho Cloudflare → Azure, dispensa desafio, funciona com Full (strict) e vale anos. O certificado que o **usuário** vê é o público do Cloudflare — o Origin CA nunca aparece para o navegador (e por isso não precisa ser confiável por ele).

## 10. O primeiro deploy

```
GHCR_USER=seu-usuario AZURE_VM_IP=1.2.3.4 APP_HOST=seunome.capacita.exemplo.tech \
bundle exec kamal config
```

Isso valida o `deploy.yml` sem tocar em nada. Depois:

**Actions → Deploy (Kamal) → Run workflow → command:** `setup`

`setup` instala o Docker se faltar, sobe os accessories e faz o primeiro deploy. Só na primeira vez — depois é `deploy`.

Deu certo:

```
curl https://seunome.capacita.exemplo.tech/api/v1/status
```

```
{"status":"ok","service":"atomic-auth-api","environment":"production"}
```

A partir daqui, `git push` na `main` faz deploy sozinho.

## 11. Operar

```
export GHCR_USER=... AZURE_VM_IP=... APP_HOST=...

kamal app logs -f           # logs ao vivo
kamal app exec --interactive --reuse "bin/rails console"
kamal app exec "bin/rails db:migrate"
kamal app details           # o que está rodando
kamal accessory reboot db   # depois de mudar env do banco
kamal rollback              # volta para a versão anterior
```

Na VM:

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP
docker ps
free -h
df -h
```

### Backup

**Volume Docker não é backup.** Se a VM sumir, o volume some junto.

```
ssh azureuser@SEU_IP \
'docker exec atomic-auth-api-db pg_dump -U atomic_auth_api atomic_auth_api_production' \
| gzip > backup-$(date +%F).sql.gz
```

E, o que quase ninguém faz: **restaure num banco de teste**. Backup que nunca foi restaurado não é backup, é esperança.

---

## Exercício da aula

---

1. Crie a VM na Azure, sem porta pública nenhuma.
  2. Abra a 22 só para o seu IP, mais a 80 e a 443.
  3. Instale o Docker e rode o `hello-world`.
  4. Rode o `server-swap.sh` e endureça o SSH.
  5. Configure o OIDC e cadastre todos os secrets e variables.
  6. Crie o registro DNS e o certificado Origin CA. Ligue o Full (strict).
  7. Rode o workflow com `command: setup`.
  8. `curl https://seunome.capacita.<domínio>/api/v1/status`
  9. Faça o fluxo completo da Aula 3 **contra a sua API em produção**: cadastro, e-mail de verdade na sua caixa de entrada, login, logout, recuperação de senha.
  10. Mude a mensagem da rota de status, dê push na `main` e veja o deploy acontecer sozinho.
- 

## Recapitulando

---

- VPS: você controla tudo, e opera tudo.
  - Escolha a ferramenta do tamanho do problema. Kubernetes não era o tamanho.
  - Imagem é receita, container é bolo. Multi-stage e usuário não-root.
  - `clear` é configuração, `secret` é segredo — e a diferença aparece no `docker inspect`.
  - Volume é o que faz o dado sobreviver ao deploy. E ainda assim não é backup.
  - OIDC troca segredo de longa duração por token de curta. Prefira sempre.
  - A porta 22 abre por um minuto e fecha — inclusive quando dá errado.
  - Full (strict), sempre. `Flexible` mente para o usuário.
- 

## O que o `seem-backend` tem a mais

---

O que ficou de fora daqui, e por quê:

| Recurso                     | Para quê                                                                           |
|-----------------------------|------------------------------------------------------------------------------------|
| <b>Datadog</b> (logs)       | logs centralizados e Error Tracking, com o app logando JSON estruturado            |
| <b>rack-attack</b>          | bloqueia scanner e rajada de erro por IP — a internet bate na sua porta o dia todo |
| <b>Active Storage</b>       | foto de perfil, em volume Docker persistente                                       |
| <b>Solid Queue</b> dedicado | processamento em background                                                        |
| <b>Audit log</b>            | histórico de toda mutação feita por admin                                          |
| <b>Export .xlsx</b>         | relatório de presença                                                              |
| <b>OpenAPI/Swagger</b>      | contrato da API documentado                                                        |

Nenhum deles é difícil depois do que você viu aqui. Todos estão no [seem-backend](#) — agora dá para ler.

# Ruby para quem sabe Python

Você já sabe programar. Isto aqui é tradução, não curso de lógica.

## Sintaxe

| Python                                                     | Ruby                                                             |
|------------------------------------------------------------|------------------------------------------------------------------|
| Indentação delimita bloco                                  | <code>def ... end</code>                                         |
| <code>None</code> / <code>True</code> / <code>False</code> | <code>nil</code> / <code>true</code> / <code>false</code>        |
| <code>snake_case</code> para funções                       | <code>snake_case</code> (igual)                                  |
| <code>PascalCase</code> para classes                       | <code>PascalCase</code> (igual)                                  |
| <code>CONSTANTE</code> por convenção                       | Constante é qualquer nome com inicial maiúscula                  |
| <code>f"Olá {nome}"</code>                                 | <code>"Olá #{nome}"</code>                                       |
| <code># comentário</code>                                  | <code># comentário</code>                                        |
| <code>return</code> explícito                              | A última expressão é o retorno                                   |
| <code>and</code> / <code>or</code> / <code>not</code>      | <code>&amp;&amp;</code> / <code>&amp; \ </code> / <code>!</code> |
| <code>elif</code>                                          | <code>elsif</code>                                               |
| <code>len(x)</code>                                        | <code>x.length</code>                                            |
| <code>str(x)</code>                                        | <code>x.to_s</code>                                              |
| <code>int("3")</code>                                      | <code>"3".to_i</code>                                            |
| <code>print(x)</code>                                      | <code>puts x</code>                                              |

## Só em Ruby

```
faca_algo if condicao      # modificador: if no fim da linha
faca_algo unless condicao  # unless = if not
5.times { |i| puts i }    # número é objeto
nome = usuario&.nome      # safe navigation (o ?. de outras linguagens)
valor ||= "padrao"        # atribui só se estiver nil/false
```

## Aspas: a diferença que Python não tem

Em Python, `'a'` e `"a"` são a mesma coisa. **Em Ruby, não:**

```
"Olá, #{nome}"      # interpola
'Olá, #{nome}'      # imprime literalmente #{nome}
```

Aspas simples são texto cru. Use aspas duplas por padrão; simples quando o texto tiver `#{` ou muitas barras invertidas.

## Números: a pegadinha da divisão

```
7 / 2      # 3.5    em Python 3
7 // 2     # 3
```

```
7 / 2      # 3      - inteiro dividido por inteiro dá inteiro
7.0 / 2    # 3.5
7.fdiv(2)  # 3.5
```

**Python 3 mudou esse comportamento; Ruby não.** É a origem de um bug silencioso quando se calcula média ou porcentagem.

## Condicionais e `case`

```
# Python
if x == 1:
    ...
elif x == 2:
    ...
else:
    ...

match x:                                # Python 3.10+
    case 1: ...
    case 2: ...
```

```
# Ruby
if x == 1
    ...
elsif x == 2
    ...
else
    ...
end

case x
when 1 then ...
when 2 then ...
else ...
end
```

O `then` permite a linha única. O `case` de Ruby também aceita faixa, classe e regex:

```
case idade
when 0..12 then "criança"
when 13..17 then "adolescente"
else
  "adulto"
end
```

E, ao contrário de `switch` em C ou Java, **não existe** `break`: cada `when` já para sozinho.

Isso aparece no `SessionsController`:

```
case session_params[:client]
when "mobile" then response.headers["Authorization"] = "Bearer #{token}"
when "web" then set_auth_cookie(token)
end
```

## Ternário e loops

```
mensagem = ativo ? "sim" : "não"      # em Python: "sim" if ativo else "não"

while restam > 0 ... end
until pronto ... end                  # o contrário do while – não existe em Python
loop do ... break if pronto ... end
```

### `puts`, `print` e `p`

| Ruby                 | Faz                                                                             |
|----------------------|---------------------------------------------------------------------------------|
| <code>puts x</code>  | imprime com quebra de linha. É o <code>print()</code> do dia a dia.             |
| <code>print x</code> | imprime sem quebra de linha                                                     |
| <code>p x</code>     | imprime a <b>representação</b> (com aspas, colchetes) e <b>devolve o objeto</b> |

`p` é o que você usa para depurar: `p usuario` mostra o objeto inteiro, e como ele devolve o valor, dá para enfiar no meio de uma expressão sem quebrar nada.

### `require` × `import`

```
import json          # Python
from pathlib import Path
```

```
require "json"        # Ruby: biblioteca ou gem
require_relative "helper" # arquivo ao lado, caminho relativo
```

`require` carrega uma vez só e devolve `false` se já estava carregado. **Não** traz nomes para o escopo como o `from x import y` — depois do `require`, você usa `JSON.parse` com o nome completo.

Dentro do Rails você nunca escreve `require`. O Zeitwerk carrega a classe pelo nome do arquivo. Fora do Rails — num script solto — o `require` volta a ser necessário.

## Splat

```
def f(*args, **kwargs): ...
```

```
def f(*args, **opts); end
```

```
valores = [1, 2, 3]
```

```
f(*valores) # espalha o array nos argumentos
```

## Estruturas de dados

| Python                     | Ruby                                                      |
|----------------------------|-----------------------------------------------------------|
| <code>[1, 2, 3]</code>     | <code>[1, 2, 3]</code> — <code>Array</code>               |
| <code>{"a": 1}</code>      | <code>{ a: 1 }</code> — <code>Hash</code> , chave símbolo |
| <code>{"a": 1}["a"]</code> | <code>{ a: 1 }[:a]</code>                                 |
| <code>(1, 2)</code> tupla  | não existe (use array congelado)                          |
| <code>{1, 2}</code> set    | <code>Set.new([1, 2])</code>                              |
| não tem                    | <code>:simbolo</code>                                     |

```
usuario = { nome: "Ana", role: :admin }
usuario[:nome] # "Ana"
usuario.fetch(:idade, 0) # 0 — como dict.get com padrão
```

## Símbolos

Um símbolo é uma **etiqueta**: um texto usado como nome, não como conteúdo. O mesmo símbolo é sempre o mesmo objeto na memória, o que o torna barato de comparar.

```
"admin" == "admin" # true, mas são dois objetos diferentes
:admin == :admin   # true, e é o mesmo objeto
```

**Regra prática:** conteúdo que o usuário lê → string. Nome de coisa no código → símbolo.

## Blocos, o coração do Ruby

Um bloco é código que você entrega para um método executar. Em Python isso é o `lambda`, usado de vez em quando. Em Ruby é o caminho normal.

```
# Python
for u in usuarios:
    print(u.nome)

nomes = [u.nome for u in usuarios]
ativos = [u for u in usuarios if u.ativo]
total = sum(u.pontos for u in usuarios)
```

```
# Ruby
usuarios.each do |u|
  puts u.nome
end

nomes = usuarios.map { |u| u.nome }
ativos = usuarios.select { |u| u.ativo? }
total = usuarios.sum { |u| u.pontos }
```

Chaves quando cabe numa linha, `do ... end` quando não cabe.

### Bloco, proc e lambda

Bloco não é objeto: ele existe só na chamada do método. Quando você quer **guardar** o código numa variável ou passar adiante, ele vira `Proc` ou `lambda`.

```
dobro = lambda x: x * 2      # Python
dobro(3)
```

```
dobro = ->(x) { x * 2 }      # Ruby: a "seta" é um lambda
dobro.call(3)
dobro.(3)                    # açúcar
```

A seta `->` é o que você vai ver no código deste projeto:

```
scope :recentes, -> { order(occurred_at: :desc) }
validate :password_meets_policy, if: -> { password.present? }
```

Nos dois casos o Rails guarda aquele pedaço de código para executar **depois** — na hora da consulta ou na hora da validação. Por isso precisa ser um objeto, e não um bloco.

`->` e `Proc.new` são quase a mesma coisa. A diferença que importa: o `lambda` confere o número de argumentos e o `return` dele volta só do `lambda`. O `Proc` é relaxado nos dois pontos. Na dúvida, use `->`.

## Equivalências

| Python                                                  | Ruby                                                         |
|---------------------------------------------------------|--------------------------------------------------------------|
| list comprehension                                      | <code>.map</code>                                            |
| <code>filter</code> / comprehension com <code>if</code> | <code>.select</code> (e <code>.reject</code> para o inverso) |
| <code>functools.reduce</code>                           | <code>.reduce</code> / <code>.inject</code>                  |
| <code>any()</code> / <code>all()</code>                 | <code>.any?</code> / <code>.all?</code>                      |
| <code>sorted(x, key=...)</code>                         | <code>x.sort_by { ... }</code>                               |
| <code>enumerate</code>                                  | <code>.each_with_index</code>                                |
| <code>zip</code>                                        | <code>.zip</code>                                            |
| <code>next(x for x in l if ...)</code>                  | <code>.find { ... }</code>                                   |

## Argumentos nomeados

```
# Python
def login(email, password, client="mobile"): ...
login(email="a@b.c", password="x")
```

```
# Ruby – os dois-pontos DEPOIS do nome tornam obrigatório nomear na chamada
def login(email:, password:, client: "mobile")
  end

login(email: "a@b.c", password: "x")
```

Sem os dois-pontos, é argumento posicional como em Python. Com eles, quem chama **tem** que nomear — e a ordem deixa de importar.

### O atalho do Ruby 3.1

Quando a variável tem o mesmo nome da chave, você omite o valor:

```
email = "a@b.c"
password = "x"

login(email:, password:)      # em vez de login(email: email, password: password)
```

Isso aparece em **todo service do projeto**:

```
Result.new(success?: true, user:)    # user: user
```

Não é erro de digitação.

---

## Struct

---

Quando você só quer agrupar valores, sem comportamento:

```
Result = Struct.new(:success?, :user, :errors, keyword_init: true)

r = Result.new(success?: true, user: ana)
r.success?    # true
r.user        # ana
```

`keyword_init: true` obriga a nomear na criação. É o `namedtuple` / `dataclass` do Python. Todo service deste projeto devolve um `Struct` desses.

---

## Exceções

---

| Python                                      | Ruby                       |
|---------------------------------------------|----------------------------|
| <code>try</code>                            | <code>begin</code>         |
| <code>except</code>                         | <code>rescue</code>        |
| <code>finally</code>                        | <code>ensure</code>        |
| <code>raise</code>                          | <code>raise</code>         |
| <code>Exception</code> (base para capturar) | <code>StandardError</code> |

```
try:
    arriscado()
except ValueError as e:
    print(e)
finally:
    limpar()
```

```
begin
  arriscado
rescue ArgumentError => e
  puts e.message
ensure
  limpar
end
```

Dentro de um método, o `begin` é implícito — dá para escrever só o `rescue` no fim:

```
def call
  arriscado
rescue StandardError => e
  Rails.error.report(e)
  nil
end
```

Criando o seu tipo de erro:

```
class InvalidToken < StandardError; end

raise InvalidToken if token_ruim?
```

Herde sempre de `StandardError`, nunca de `Exception`. `rescue` sem classe captura `StandardError`; capturar `Exception` pega até `Ctrl+C` e erro de falta de memória.

`raise` sem argumento, dentro de um `rescue`, relança a exceção atual.

## Atalhos de literal

```
%w[mobile web]      # => ["mobile", "web"]    – array de strings
%i[name email]       # => [:name, :email]    – array de símbolos
```

Só economizam aspas e vírgulas. Aparecem em constantes por todo o projeto:

```
CLIENTS = %w[mobile web].freeze
REQUIRED_FIELDS = %i[name email password].freeze
```

`.freeze` congela o objeto: tentar alterar depois levanta erro. Em Ruby strings e arrays são mutáveis (diferente de Python), então constante sem `freeze` é constante só no nome.

## Classes

```
class Usuario:
    def __init__(self, nome):
        self.nome = nome

    def saudacao(self):
        return f"Oi, {self.nome}"

    @property
    def admin(self):
        return self.role == "admin"
```

```
class Usuario
  attr_accessor :nome          # gera o getter e o setter

  def initialize(nome)
    @nome = nome              # @ marca variável de instância
  end

  def saudacao
    "Oi, #{@nome}"
  end

  def admin?
    role == "admin"
  end
end
```

| Python                                    | Ruby                                      |
|-------------------------------------------|-------------------------------------------|
| <code>__init__</code>                     | <code>initialize</code>                   |
| <code>self.x</code>                       | <code>@x</code>                           |
| <code>self</code> como primeiro parâmetro | <code>self</code> implícito               |
| <code>@property</code>                    | método normal (não precisa de parênteses) |
| <code>@staticmethod</code>                | <code>def self.metodo</code>              |
| <code>_privado</code> por convenção       | <code>private</code> de verdade           |
| herança múltipla                          | módulos ( <code>include</code> )          |

? e !

```
user.admin?    # devolve true/false
user.save      # devolve false se falhar
user.save!     # estoura ActiveRecord::RecordInvalid se falhar
```

Convenção, não regra da linguagem. Mas todo mundo segue — e o Rails inteiro depende dela.

---

## Módulos e mixins

---

Ruby não tem herança múltipla. Tem módulo: um saco de métodos que você inclui.

```
# Python: mixin por herança múltipla
class Usuario(Base, VerificavelPorEmail, RecuperaSenha):
    pass
```

```
# Ruby: include
class Usuario < ApplicationRecord
  include EmailVerifiable
  include PasswordResetable
end
```

```
module EmailVerifiable
  extend ActiveSupport::Concern

  def email_verificado?
    email_verified_at.present?
  end
end
```

No Rails, um módulo desses vive em `app/models/concerns/` e se chama **concern**. É o que impede o `User` de virar um arquivo de 800 linhas.

---

## Dependências e ferramentas

| Python                                        | Ruby                                                          |
|-----------------------------------------------|---------------------------------------------------------------|
| <code>pip install x</code>                    | <code>gem install x</code>                                    |
| <code>requirements.txt</code>                 | <code>Gemfile</code>                                          |
| <code>pip freeze &gt; requirements.txt</code> | <code>Gemfile.lock</code> (gerado sozinho, sempre versionado) |
| <code>venv</code> / <code>virtualenv</code>   | <code>bundler</code> isola por projeto                        |
| <code>pyenv</code>                            | <code>mise</code> / <code>rbenv</code>                        |
| <code>python -m x</code>                      | <code>bundle exec x</code>                                    |
| <code>pytest</code>                           | <code>minitest</code> (padrão do Rails) ou <code>rspec</code> |
| <code>black</code> / <code>ruff</code>        | <code>rubocop</code>                                          |
| <code>mypy</code>                             | não há equivalente padrão — Ruby é dinâmica de ponta a ponta  |
| <code>python</code> (REPL)                    | <code>irb</code>                                              |

## Rails × Django

| Django                                                | Rails                                                                  |
|-------------------------------------------------------|------------------------------------------------------------------------|
| <code>models.py</code> com classes <code>Model</code> | <code>app/models/</code> , um arquivo por classe                       |
| <code>makemigrations</code> / <code>migrate</code>    | <code>rails g migration</code> / <code>rails db:migrate</code>         |
| <code>urls.py</code> , escrito à mão                  | <code>config/routes.rb</code>                                          |
| <code>views.py</code>                                 | <code>app/controllers/</code>                                          |
| <code>serializers.py</code> (DRF)                     | <code>app/serializers/</code> (feito à mão neste projeto)              |
| Django ORM: <code>User.objects.filter(...)</code>     | ActiveRecord: <code>User.where(...)</code>                             |
| <code>manage.py</code>                                | <code>bin/rails</code>                                                 |
| <code>settings.py</code>                              | <code>config/application.rb</code> + <code>config/environments/</code> |
| <code>.env</code> + <code>django-environ</code>       | <code>Rails.application.credentials</code> + ENV                       |
| <code>python manage.py shell</code>                   | <code>bin/rails console</code>                                         |

## Consultas lado a lado

```
User.objects.filter(role="admin")
User.objects.get(id=1)
User.objects.filter(email__icontains="ufop").order_by("-created_at")[:10]
User.objects.count()
```

```
User.where(role: "admin")
User.find(1)
User.where("email ILIKE ?", "%ufop%").order(created_at: :desc).limit(10)
User.count
```

## Pegadinhas de quem vem do Python

| Pegadinha                                                                                                                | O que acontece                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>0</code> e <code>""</code> são <b>verdadeiros</b> em Ruby                                                          | Só <code>nil</code> e <code>false</code> são falsos. <code>if 0</code> executa.                  |
| <code>puts</code> devolve <code>nil</code>                                                                               | Não use o retorno de <code>puts</code>                                                           |
| Parênteses são opcionais                                                                                                 | <code>user.save</code> e <code>user.save()</code> são a mesma coisa                              |
| <code>=</code> no fim do nome define setter                                                                              | <code>def nome=(valor)</code> habilita <code>obj.nome = "x"</code>                               |
| Strings são mutáveis                                                                                                     | Diferente de Python. Use <code>.freeze</code> em constantes.                                     |
| <code>7 / 2</code> dá <code>3</code> , não <code>3.5</code>                                                              | Python 3 mudou isso; Ruby não. Use <code>7.0 / 2</code> ou <code>7.fdiv(2)</code> .              |
| <code>'texto #{x}'</code> não interpola                                                                                  | Só aspas duplas interpolam.                                                                      |
| <code>case</code> não precisa de <code>break</code>                                                                      | Cada <code>when</code> já para sozinho.                                                          |
| <code>and</code> / <code>or</code> existem, mas têm precedência diferente de <code>&amp;&amp;</code> / <code>\ \ </code> | Use <code>&amp;&amp;</code> e <code>\ \ </code> sempre                                           |
| Não existe <code>elif</code>                                                                                             | É <code>elsif</code>                                                                             |
| Range com <code>..</code> inclui o fim, com <code>...</code> não                                                         | <code>(1..3).to_a</code> → <code>[1,2,3]</code> ; <code>(1...3).to_a</code> → <code>[1,2]</code> |

# Glossário

---

Termos que aparecem nas quatro aulas, em ordem alfabética.

---

**Accessory** — Container que o Kamal gerencia mas não faz deploy junto com a aplicação. O Postgres é um. `kamal deploy` não sobe accessory; use `kamal accessory boot|reboot`.

**ActiveRecord** — O ORM do Rails. Cada classe é uma tabela, cada objeto é uma linha. Equivalente ao Django ORM.

**API** — *Application Programming Interface*. A porta de entrada do sistema para outros programas. Aqui, uma API REST que responde JSON.

**Associação** — Ligação entre dois models. `has_many` no lado "um", `belongs_to` no lado "muitos" — que é quem carrega a chave estrangeira.

**Base64** — Forma de escrever bytes usando só letras e números, para caber em texto. **Não é criptografia**: não tem chave e qualquer um desfaz. O JWT é Base64.

**Bcrypt** — Função de hash feita para senha. Lenta de propósito, com fator de custo ajustável e salt aleatório por senha. Ver `02-activerecord.md`.

**Bearer** — O esquema do cabeçalho `Authorization: Bearer <token>`. "Portador": quem apresenta o token é tratado como o dono dele.

`before_action` — Filtro que roda antes da action do controller. Se ele renderizar algo, a action não é chamada.

**Brakeman** — Análise estática de segurança para Rails. Procura SQL injection, mass assignment, redirect aberto e afins.

**Bundler** — O gerenciador de dependências do Ruby. Lê o `Gemfile`, resolve versões e escreve o `Gemfile.lock`.

**CA (Autoridade Certificadora)** — Quem assina certificados. O navegador nasce com uma lista de CAs em que confia; confiar na CA é confiar em quem ela assinou.

**Certificado** — Documento que afirma "esta chave pública pertence a este domínio", assinado por uma CA.

**Chave estrangeira** — Coluna que aponta para o `id` de outra tabela (`login_events.user_id`). Com `foreign_key: true`, o banco recusa linha órfã.

**CI/CD** — *Continuous Integration / Continuous Deployment*. CI roda testes e verificações a cada push; CD publica automaticamente o que passou.

**Concern** — Módulo Ruby que uma classe inclui para ganhar um conjunto de métodos. É o mixin do Rails. Vive em `app/models/concerns/` ou `app/controllers/concerns/`.

**Container** — Uma instância rodando de uma imagem Docker. Descartável.

**Cookie** — Par nome=valor que o servidor manda no `Set-Cookie` e o navegador reenvia sozinho em toda requisição àquele site. É a memória que o HTTP não tem, colada por fora.

**CORS** — *Cross-Origin Resource Sharing*. A regra do navegador que impede uma página de um domínio chamar uma API de outro sem permissão explícita. Não se aplica a app nativo.

**Credentials** — Arquivo YAML criptografado do Rails (`config/credentials.yml.enc`), decriptado pela `master.key`.

**Criptografia assimétrica** — Duas chaves que se completam: a pública se espalha, a privada nunca sai da máquina. Base do SSH e do TLS.

**CSRF** — *Cross-Site Request Forgery*. Site malicioso dispara requisição para a sua API, e o navegador anexa o cookie da vítima. Mitigado com `same_site`.

**CVE** — *Common Vulnerabilities and Exposures*. O identificador público de uma vulnerabilidade conhecida. `bundler-audit` procura CVE nas suas gems.

**Denylist** — Lista de tokens revogados. No nosso logout, indexada pelo `jti` do JWT.

**Digest** — O resultado de passar um valor por uma função de hash. `password_digest` guarda o digest, nunca a senha.

**DNS** — A agenda telefônica da internet: traduz um nome (`api.exemplo.com`) no IP da máquina. A resposta fica em cache pelo TTL.

**Docker** — Empacota aplicação e dependências numa imagem que roda igual em qualquer lugar.

**Dockerfile** — A receita da imagem. O nosso é multi-stage: um estágio compila, o final só copia o resultado.

**Enumeração de usuários** — Ataque em que respostas diferentes (mensagem, status ou tempo) revelam quem tem conta no sistema. Combatido com respostas idênticas e tempo constante.

**Fixture** — Dados de teste em YAML, carregados antes de cada teste e limpos depois.

**Gem** — Biblioteca Ruby. O equivalente do pacote do pip.

**ghcr.io** — GitHub Container Registry. Onde a imagem Docker fica hospedada.

**Handshake (TLS)** — A negociação inicial: o servidor apresenta o certificado, o cliente valida a cadeia, e os dois combinam uma chave temporária.

**Health check** — Rota que responde 200 se a aplicação está de pé. A nossa é `/up`. O Kamal só troca o tráfego para o container novo depois que ela passa.

**Imagem** — A receita congelada: sistema, runtime, dependências e código. Imutável.

**IP** — O endereço da máquina na rede ( `57.156.65.151` ). `127.0.0.1` é sempre "esta máquina aqui".

**jti** — *JWT ID*. Identificador único de um token, usado para revogá-lo individualmente.

**JWT** — *JSON Web Token*. Cabeçalho, payload e assinatura, em Base64, separados por ponto. O payload é **público**; a assinatura só garante que ninguém o alterou.

**Kamal** — Ferramenta de deploy com Docker via SSH, feita pela equipe do Rails. Zero-downtime, sem agente na máquina.

**Mass assignment** — Vulnerabilidade em que o cliente manda um campo que não devia poder mandar (ex.: `role: admin`). Evitada com strong parameters.

**Middleware** — Camadas que a requisição atravessa antes de chegar ao controller. CORS, cookies e rate limit moram aí. `bin/rails middleware` lista a pilha.

**Migration** — Mudança no banco, versionada em código, que roda uma vez em cada máquina.

**Minitest** — O framework de teste que vem com o Rails.

**mise** — Gerenciador de versões de runtime. O `pyenv` do mundo Ruby.

**MVC** — *Model-View-Controller*. Model = dados e regras; View = a saída (aqui, JSON); Controller = recebe a requisição e decide.

**N+1** — Carregar uma lista e depois consultar o banco item a item. 100 registros viram 101 consultas. Resolve-se com `includes`.

**NSG** — *Network Security Group*. O firewall da Azure, com regras de entrada e saída por porta e origem.

**OIDC** — *OpenID Connect*. Permite o GitHub Actions autenticar na Azure com um token de curta duração, sem guardar segredo de longa duração.

**OOM killer** — O mecanismo do kernel Linux que mata processos quando a RAM acaba. É por isso que a VM tem swap.

**Origin CA** — Certificado emitido pelo Cloudflare para o trecho Cloudflare → seu servidor. Navegador não confia nele diretamente, e não precisa: o usuário vê o certificado público do Cloudflare.

**ORM** — *Object-Relational Mapping*. Traduz objetos em linhas de tabela.

**OTP** — *One-Time Password*. O código de 6 dígitos, válido por 15 minutos e uma vez só.

**Payload** — O miolo do JWT, onde ficam `sub`, `exp`, `jti`. Público.

**Porta** — Número de 1 a 65535 que identifica um programa dentro da máquina. 22 SSH, 80 HTTP, 443 HTTPS, 5432 Postgres, 3000 Rails em dev.

**Postgres** — O banco relacional que usamos em desenvolvimento e em produção.

**Proxied (Cloudflare)** — Nuvem laranja no DNS: o tráfego passa pelo Cloudflare antes de chegar à sua máquina, e o IP real não aparece no DNS.

**Proxy reverso** — Fica na frente do servidor: recebe na 443, termina o TLS e repassa para a aplicação em HTTP interno. O `kamal-proxy` é um.

**Puma** — O servidor web do Rails.

**rack-attack** — Middleware que bloqueia abuso por IP. Está no `seem-backend`, não no app do curso.

**Registry** — Repositório de imagens Docker. O "GitHub das imagens".

**REST** — Estilo de organizar a API: cada coisa é um recurso com um endereço, e o verbo HTTP diz o que fazer com ele.

**Rollback** — O banco desfazendo uma transação que falhou no meio. Também: `kamal rollback`, que volta para a versão anterior da imagem.

**root** — O usuário do Linux que pode tudo, sem confirmação. Você trabalha como usuário comum e chama `sudo` quando precisa.

**RuboCop** — O linter de Ruby. O `black` / `ruff` do mundo Python.

**Salt** — Valor aleatório misturado à senha antes do hash, para que senhas iguais gerem digests diferentes. O bcrypt gera um por senha, automaticamente.

**Scope** — Consulta com nome, definida no model e encadeável:

```
user.login_events.recentes.limit(5).
```

**Serializer** — Decide o que de um objeto vai para o JSON — e, principalmente, o que não vai.

**Service object** — Classe que representa um caso de uso (`Users::Register`). Tira a regra de negócio do controller.

**Solid Cache / Solid Queue** — Cache e fila do Rails 8, guardados no próprio Postgres. A denylist do logout usa o cache.

**Strong parameters** — Filtro que declara quais campos a requisição pode mandar. No Rails 8, `params.expect`.

**Struct** — Classe de uma linha para agrupar valores sem comportamento. O `namedtuple` / `dataclass` do Ruby. Todo service do projeto devolve um.

**sub** — *Subject*. O claim do JWT que diz de quem é o token — no nosso caso, o `user.id`.

**Swap** — Área em disco usada como extensão da RAM. Lenta, mas evita que o OOM killer mate a aplicação.

**TLS** — O túnel criptografado que embrulha o HTTP e o transforma em HTTPS. Mesmo protocolo, só que fechado.

**Token version** — Contador no `User` que vai dentro do JWT. Incrementar invalida todos os tokens daquele usuário de uma vez.

**Transação** — Bloco em que ou tudo acontece, ou nada acontece. `User.transaction do ... end`.

**TTL** — *Time To Live*. Por quanto tempo algo vale. Nosso código OTP tem TTL de 15 minutos; as entradas da denylist têm o TTL do que restava do token.

**Variável de ambiente** — Par nome=valor que o sistema entrega ao processo. Lida com `ENV["NOME"]`. É como configuração e segredo entram sem passar pelo código.

**Volume (Docker)** — Área de disco que sobrevive ao container. É o que faz o banco não sumir a cada deploy. **Não é backup.**

**VPS** — *Virtual Private Server*. Um computador virtual que é seu, com acesso root.

**WSL2** — *Windows Subsystem for Linux*. Um Linux de verdade dentro do Windows. Obrigatório para quem desenvolve Rails no Windows.

**XSS** — *Cross-Site Scripting*. O atacante roda JavaScript dentro da sua página, geralmente porque você exibiu texto de usuário sem escapar. É contra isso que `httponly` protege o token.

**Zeitwerk** — O autoloader do Rails. Descobre o arquivo pelo nome da classe:

`Api::V1::StatusController` → `app/controllers/api/v1/status_controller.rb`. É o que dispensa o `require`.

**Índice** — Estrutura no banco que acelera busca. Com `unique: true`, vira restrição: o banco recusa duplicata mesmo com duas requisições simultâneas.

# Troubleshooting

---

Erros que acontecem de verdade, e a saída de cada um. Os da parte de infraestrutura vieram do histórico real de deploy do `seem-backend`.

---

## Ambiente

---

`mise: command not found`

A linha de `activate` não foi para o arquivo certo. Confira o fim do `~/.bashrc` (ou `~/.zshrc`) e rode `exec bash`.

### A instalação do Ruby falha compilando

Faltou dependência de build. Rode o bloco de `apt install` / `brew install` de `00-preparacao.md` e tente `mise install ruby@3.4.9` de novo.

`gem install rails` reclama de permissão

Você está usando o Ruby do sistema, não o do mise. Confira:

```
which ruby # tem que apontar para dentro de ~/.local/share/mise
```

`permission denied` ao rodar `docker`

Você não está no grupo `docker`:

```
sudo usermod -aG docker $USER
```

E **saia e entre de novo na sessão**. Reabrir o terminal não basta.

### No Windows, `docker` não aparece dentro do Ubuntu

Docker Desktop → Settings → Resources → WSL Integration → habilite a distro `Ubuntu-24.04`.

---

## Banco

---

### `address already in use` ao subir o compose

Já existe um Postgres na sua máquina ocupando a 5432.

```
DB_PORT=5433 docker compose up -d
export DB_PORT=5433          # e exporte no shell onde roda o Rails
```

Para achar quem ocupa: `ss -ltnp | grep 5432`.

### `PG::ConnectionBad: could not connect to server`

Nesta ordem:

```
docker compose ps          # o container está "healthy"?
docker compose logs db     # o que ele diz?
echo $DB_PORT              # bate com a porta publicada?
```

### `PG::UndefinedTable` ou `PendingMigrationError`

Faltou migrar:

```
bin/rails db:migrate
```

## Quero começar do zero

```
bin/rails db:reset          # apaga, recria, carrega o schema e roda os seeds
```

---

## Testes

### Um teste passa sozinho e falha junto com os outros

Vazamento de estado entre testes. Suspeitos, em ordem:

1. um `Rails.cache` compartilhado (o `test_helper` troca por `MemoryStore` a cada teste);
2. `ActionMailer::Base.deliveries` não limpo;
3. dependência de ordem — o Minitest embaralha de propósito, e isso é uma qualidade.

Rode com a mesma semente para reproduzir: `bin/rails test --seed 1234`.

## O teste de logout passa mas a revogação não funciona

Em teste, o `Rails.cache` padrão é o `null_store`: não guarda nada, e `revoked?` sempre devolve `false`. O `test_helper.rb` troca por `MemoryStore` justamente por isso. Se você recriou o arquivo, recoloque o `setup`.

### `Bullet::Notification::UnoptimizedQueryError`

Consulta N+1: você carregou uma coleção e acessou uma associação item a item. Adicione `includes`. (O `seem-backend` usa o Bullet; o app do curso não.)

---

## Autenticação

---

### Sempre 401, mesmo com o token certo

Cheque, nesta ordem:

1. o cabeçalho é exatamente `Authorization: Bearer <token>`, com o espaço;
2. o token não foi revogado (você fez logout com ele?);
3. o `token_version` do banco bate com o `ver` do token — trocar a senha incrementa a versão;
4. o `JWT_SECRET` é o mesmo que emitiu o token. Reiniciar o app em dev sem `JWT_SECRET` definido usa o `secret_key_base`, que é estável — mas em produção um segredo diferente invalida tudo.

Decodifique o token em [jwt.io](https://jwt.io) e olhe o `exp` e o `ver`.

### O e-mail não chega em desenvolvimento

Ele não é enviado: o `letter_opener` abre no navegador. Se você está rodando `curl` e não vendo nada, o arquivo está em `tmp/letter_opener/`.

### `ActionController::ParameterMissing (400)`

Faltou um campo obrigatório no corpo. O `params.expect` exige todas as chaves declaradas. Confira o JSON e o `Content-Type: application/json`.

### 422 no cadastro sem dizer o motivo direito

Olhe `error.details` na resposta: cada item tem `field` e `message`.

---

## Azure e VM

---

### `NotAvailableForSubscription` ao criar a VM

O tamanho escolhido não está habilitado para a sua assinatura naquela região.

**Assinaturas** → **Uso + cotas** → **filtre por Compute** e escolha outra região ou outro tamanho. Pedir aumento de cota **não** resolve quando a oferta não está habilitada.

### SSH dá timeout

Quase sempre é o NSG:

1. o seu IP mudou? `curl -4 ifconfig.me` e compare com a regra;
2. a regra libera a porta 22, protocolo TCP, direção Entrada?
3. a prioridade da regra de permissão é **menor** que a de alguma negação?

### `Permission denied (publickey)`

```
chmod 600 ~/.ssh/azure-capacita
ssh -i ~/.ssh/azure-capacita -o IdentitiesOnly=yes azureuser@SEU_IP
```

O `IdentitiesOnly=yes` importa: sem ele o SSH tenta todas as chaves do agente, o servidor recusa depois de algumas e você leva `Too many authentication failures` com a chave certa na mão.

### Perdi o acesso depois de mexer no sshd

Se você seguiu o guia, tinha uma sessão aberta — desfaça por ela. Se não, sobrou o **Serial Console** do portal da Azure (menu da VM → Suporte + solução de problemas → Console Serial), que não passa pelo SSH.

Da próxima vez: `sudo sshd -t` antes de `reload`, e teste em outro terminal antes de fechar o primeiro.

---

## Deploy

---

### OIDC: `AADSTS700213`

O *subject* da credencial federada não bate com o que o GitHub emite. Ele precisa ser, caractere por caractere:

```
repo:SEU-USUARIO/SEU-REPO:ref:refs/heads/main
```

Confira maiúsculas, o nome do repositório e a branch.

## `az: command not found` ou falha transitória do Azure CLI

O `azure/login` às vezes falha por instabilidade. Rode o workflow de novo antes de investigar.

## A regra temporária ficou no NSG

O passo de remoção tem `if: always()`, mas se o job for cancelado à força a regra pode sobrar. Apague na mão:

```
az network nsg rule delete \  
  --resource-group SEU_RG --nsg-name SEU_NSG \  
  --name allow-github-actions-deploy-ssh
```

E confira depois de todo deploy que falhou de forma estranha.

## Kamal: `Docker is not installed`

O primeiro deploy precisa ser `setup`, não `deploy`. Rode o workflow por **Actions** → **Run workflow** → **command: setup**.

## A aplicação sobe mas não acha o banco

`kamal deploy` **não** sobe accessories. O workflow tem um passo `kamal accessory boot db`, mas se você rodou o Kamal à mão:

```
kamal accessory boot db
```

Depois de mudar env de accessory, `boot` não basta — é `kamal accessory reboot db`.

## Cloudflare 521 / 522

O Cloudflare não conseguiu falar com o seu servidor.

- a porta 443 está aberta no NSG?
- `docker ps` na VM mostra o `kamal-proxy` rodando?
- o registro A aponta para o IP certo?

## Cloudflare 525 / 526

Falha no handshake TLS entre Cloudflare e a sua VM.

- **525**: o proxy não apresentou certificado. Os secrets `KAMAL_PROXY_SSL_*` estão cadastrados?
- **526**: o certificado é inválido para o modo Full (strict). Ele cobre o hostname exato? Um wildcard `*.capacita.exemplo.tech` cobre `seunome.capacita.exemplo.tech`, mas **não** cobre `x.y.capacita.exemplo.tech`.

## Let's Encrypt falha atrás do Cloudflare

Esperado. Com o DNS proxied, o desafio HTTP-01 não chega como o Let's Encrypt espera. Use o certificado Origin CA — é a razão de ele existir.

## SMTP dá timeout na VM

Vários provedores de nuvem bloqueiam a saída na porta 25, e alguns na 587. Teste a alternativa do seu provedor de e-mail (o Resend aceita 2587) e ajuste `SMTP_PORT` no `env.clear`.

## O deploy passa mas o site responde 500

```
kamal app logs -f
```

Suspeitos comuns: migration pendente ( `kamal app exec "bin/rails db:migrate"` ), `RAILS_MASTER_KEY` errada, ou um `ENV.fetch` sem default para uma variável que ninguém cadastrou.

## Preciso voltar atrás agora

```
kamal rollback
```

Volta para a versão anterior da imagem, que ainda está na VM.